

VibeVolts Simulation Toolkit

Literate Programming Source Documentation

VibeVolts Development Team

August 8, 2026

Abstract

This document presents the complete literate source code of the **VibeVolts** space environment simulation toolkit. VibeVolts models space-based and ground-based optical sensors, celestial motion, satellite dynamics, target radiometry, and discrete-event scan scheduling. In accordance with literate programming principles, every module's source code is surrounded by mathematical derivations, system architecture explanations, and hyperlinked cross-references.

Contents

1	Architecture & Foundations	2
1.1	System Overview	2
2	minimalsimulation.py: Core Simulation State Infrastructure	3
3	constants.py: Physical & Array Constants	9
4	detector.py: Optoelectronic Detector Array	10
5	propagation.py: Satellite Orbit Propagation	15
6	observatories.py: Ground Station Observatories	18
7	celestialbodies.py: Celestial Bodies Ephemeris	21
8	targets.py: Reference Target Generation	23
9	exclusion.py: Celestial Obstruction & Exclusion Zones	24
10	pointing.py: Sensor Pointing & Re-slewing	28
11	scandetectors.py: Target Detection & Photometric Engine	33
12	radiometry_data.py: Filter Spectral Data	38
13	radiometry_calcs.py: Radiometric Calculations & Blackbody	41
14	lambertian.py: Lambertian Sphere Brightness	44
15	dataHandling.py: Simulation Data Logging & Analysis	48
16	cadenceController.py: Discrete Event Scan Scheduler	52
17	fibonacciSearch.py: Fibonacci Pointing Vector Resorting	54
18	generate_log_spherical_points.py: Logarithmic Radial Distribution	57
19	plotting_3d.py: Plotly 3D Position Renderer	59
20	plotting_vectors.py: Plotly Pointing Vector Renderer	61
21	sim_report.py: Simulation State Reporting	63
22	sim_check.py: Simulation Data Verification	66
23	all_demos.py: Demonstration Suite Runner	69

Chapter 1

Architecture & Foundations

1.1 System Overview

The VibeVolts architecture is built around a centralized state container (`sim_data`, an instance of `SimulationState`). The state encapsulates:

1. **Kinematics:** Positions ($\mathbf{r} \in \mathbb{R}^3$), velocities ($\mathbf{v} \in \mathbb{R}^3$), and accelerations ($\mathbf{a} \in \mathbb{R}^3$) in the Geocentric Celestial Reference System (GCRS).
2. **Assets:** Satellites (`SatellitesState`), Ground Stations (`ObservatoriesState`), Celestial Bodies (`CelestialState`), and Stationary Targets (`FixedPointsState`).
3. **Sensors:** Parallelized optoelectronic parameters and pointing vectors (`DetectorArray`).
4. **Discrete Event Scheduler:** Cadence scheduling (`CadenceState`) organizing detectors by integration time.

Chapter 2

minimalsimulation.py: Core Simulation State Infrastructure

Overview

Defines SimulationState, ComponentState, DotDict, SchemaDict, and get_detector_positions().

Literate Source Code

The source code of `minimalsimulation.py` is documented below. Section label: [2](#).

```
1 import numpy as np
2 from datetime import datetime, timezone
3 from typing import Dict, Any, Optional, List
4 from dataclasses import dataclass, field
5 from constants import *
6
7
8 def safe_normalize(v: np.ndarray, axis: int = -1) -> np.ndarray:
9     """
10     Safely normalizes an array of vectors along the specified axis,
11     preventing division by zero.
12     """
13     norms = np.linalg.norm(v, axis=axis, keepdims=True)
14     return np.where(norms == 0, 0.0, v / norms)
15
16
17 class DotDict(Dict):
18     """
19     A clean dictionary subclass that supports dot-notation attribute access.
20     Unifies attribute access and dictionary keys into a single storage mechanism.
21     """
22     def __getattr__(self, name):
23         try:
24             return self[name]
25         except KeyError:
26             raise AttributeError(f"'{self.__class__.__name__}' object has no attribute '{name}'")
27
28     def __setattr__(self, name, value):
29         self[name] = value
30
31     def get(self, key, default=None):
32         try:
33             return self[key]
34         except (KeyError, AttributeError):
35             return default
36
37
38 class SchemaDict(DotDict):
39     """
40     A DotDict that ensures default attributes are always initialized
```

```

41 while seamlessly accepting any extra user-defined keys.
42 """
43 def __init__(self, **kwargs):
44     super().__init__(**kwargs)
45
46
47 class ComponentState(SchemaDict):
48     """
49     Base class for sub-states that represent collections of assets/components
50     stored as parallel NumPy arrays and lists.
51
52     Provides standard utility methods:
53     - len(state): returns length based on NumPy array or list dimensions.
54     - state.subset(mask): returns a sliced copy of the component state.
55     - state.append(other_state): appends another component state in-place.
56     """
57
58     def __len__(self) -> int:
59         for val in self.values():
60             if isinstance(val, np.ndarray):
61                 return val.shape[0] if val.ndim >= 1 else 0
62             elif isinstance(val, list):
63                 return len(val)
64         return 0
65
66     def subset(self, mask: Any) -> "ComponentState":
67         """
68         Slices all 1D/2D NumPy array fields and list fields by the given mask
69         and returns a new instance of the same class.
70         """
71         cls = self.__class__
72         new_kwargs = {}
73         n_total = len(self)
74
75         for key, val in self.items():
76             if isinstance(val, np.ndarray):
77                 if val.ndim == 1:
78                     new_kwargs[key] = val[mask]
79                 elif val.ndim == 2:
80                     if val.shape[0] == n_total:
81                         new_kwargs[key] = val[mask, :]
82                     elif val.shape[1] == n_total:
83                         new_kwargs[key] = val[:, mask]
84                     else:
85                         new_kwargs[key] = val[mask]
86                 else:
87                     new_kwargs[key] = val[mask]
88             elif isinstance(val, list):
89                 if isinstance(mask, np.ndarray) and mask.dtype == bool:
90                     indices = np.where(mask)[0]
91                 elif isinstance(mask, slice):
92                     indices = range(*mask.indices(len(val)))
93                 else:
94                     indices = mask
95                 new_kwargs[key] = [val[i] for i in indices]
96             else:
97                 new_kwargs[key] = val
98
99         return cls(**new_kwargs)
100
101     def append(self, other: "ComponentState") -> None:
102         """
103         Appends all matching NumPy arrays and list attributes from 'other' in-place.
104         """
105         for key, val in other.items():
106             if key in self:
107                 target = self[key]
108                 if isinstance(target, np.ndarray) and isinstance(val, np.ndarray):
109                     if target.ndim == 1:
110                         self[key] = np.append(target, val)
111                     elif target.ndim == 2:
112                         if target.shape[1] == val.shape[1]:
113                             self[key] = np.vstack([target, val])

```

```

114         elif target.shape[0] == val.shape[0]:
115             self[key] = np.hstack([target, val])
116         else:
117             self[key] = np.vstack([target, val])
118     elif isinstance(target, list) and isinstance(val, list):
119         target.extend(val)
120
121
122 class CountsState(SchemaDict):
123     """
124     CountsState provides access to asset counts.
125     If sim_data reference is available, it dynamically returns
126     the actual length of component arrays.
127     """
128     def __init__(self, parent_sim=None, **kwargs):
129         defaults = {
130             'celestial': 0,
131             'satellites': 0,
132             'observatories': 0,
133             'fixedpoints': 0,
134         }
135         dict.__setattr__(self, '_parent_sim', parent_sim)
136         super().__init__(**{**defaults, **kwargs})
137
138     def __getitem__(self, item):
139         if hasattr(self, '_parent_sim') and self._parent_sim is not None:
140             comp = self._parent_sim.get(item)
141             if comp is not None and hasattr(comp, '__len__'):
142                 return len(comp)
143             return super().__getitem__(item)
144
145     def __getattr__(self, name):
146         if name.startswith('_'):
147             return super().__getattr__(name)
148         if hasattr(self, '_parent_sim') and self._parent_sim is not None:
149             comp = self._parent_sim.get(name)
150             if comp is not None and hasattr(comp, '__len__'):
151                 return len(comp)
152             return super().__getattr__(name)
153
154     def items(self):
155         return [(k, self[k]) for k in self.keys() if not k.startswith('_')]
156
157     def values(self):
158         return [self[k] for k in self.keys() if not k.startswith('_')]
159
160
161 class CelestialState(ComponentState):
162     def __init__(self, **kwargs):
163         defaults = {
164             'position': np.zeros((2, 3)),
165             'velocity': np.zeros((2, 3)),
166             'acceleration': np.zeros((2, 3)),
167         }
168         super().__init__(**{**defaults, **kwargs})
169
170
171 class SatellitesState(ComponentState):
172     def __init__(self, **kwargs):
173         defaults = {
174             'position': np.zeros((0, 3)),
175             'velocity': np.zeros((0, 3)),
176             'acceleration': np.zeros((0, 3)),
177             'orbital_elements': np.zeros((0, 6)),
178             'epochs': [],
179         }
180         super().__init__(**{**defaults, **kwargs})
181
182
183 class FixedPointsState(ComponentState):
184     def __init__(self, **kwargs):
185         defaults = {
186             'position': np.zeros((0, 3)),

```

```

187         'exclusion': np.zeros((0,)),
188         'size': np.zeros((0,)),
189         'albedo': np.zeros((0,)),
190     }
191     super().__init__(**defaults, **kwargs)
192
193     def add_target(self, position: np.ndarray, size: float, albedo: float = 0.2) -> None:
194         pos = np.asarray(position, dtype=float).reshape(-1, 3)
195         n_new = pos.shape[0]
196         self.position = np.vstack([self.position, pos]) if self.position.size else pos
197         self.exclusion = np.append(self.exclusion, np.zeros(n_new, dtype=int))
198         self.size = np.append(self.size, np.full(n_new, size, dtype=float))
199         self.albedo = np.append(self.albedo, np.full(n_new, albedo, dtype=float))
200
201
202     class ObservatoriesState(ComponentState):
203         def __init__(self, **kwargs):
204             defaults = {
205                 'position': np.zeros((0, 3)),
206                 'velocity': np.zeros((0, 3)),
207                 'acceleration': np.zeros((0, 3)),
208                 'pointing': np.zeros((0, 3)),
209                 'latitude': np.zeros(0),
210                 'longitude': np.zeros(0),
211                 'altitude': np.zeros(0),
212             }
213             super().__init__(**defaults, **kwargs)
214
215
216     @dataclass
217     class CadenceGroup:
218         """
219         Holds scheduling information for one group of detectors that share
220         an identical integration time.
221
222         Attributes:
223             scanInterval: Integration period in seconds for this group.
224             scanMask: Boolean array (length = total detectors) selecting
225                       the detectors that belong to this group.
226             scanNext: The datetime at which the next scan for this group
227                      is due. Initialised to sim_data.time so that every
228                      group fires on the first call to nextIntegration.
229         """
230         scanInterval: float
231         scanMask: np.ndarray
232         scanNext: datetime
233
234
235     class CadenceState(SchemaDict):
236         """
237         Top-level cadence schedule stored in sim_data.cadenceStructure.
238
239         Attributes:
240             cadenceList: Ordered list of CadenceGroup objects, one per unique
241                       integration time found in sim_data.detector.
242             nextTime: The datetime of the earliest upcoming scan across all
243                     groups.
244             nextGroup: Index into cadenceList identifying which group fires
245                       next.
246         """
247         def __init__(self, **kwargs):
248             defaults = {
249                 'cadenceList': [],
250                 'nextTime': None,
251                 'nextGroup': 0,
252             }
253             super().__init__(**defaults, **kwargs)
254
255
256     class SimulationState(SchemaDict):
257         """
258         SimulationState models the main simulation data structure.

```



```

259 It inherits from SchemaDict to maintain 100% backward compatibility
260 with dict-style lookup/subscription and isinstance(..., dict) checks,
261 but supports attribute-style access for better structure.
262 """
263 _FIELD_SCHEMAS = {
264     'counts': CountsState,
265     'celestial': CelestialState,
266     'satellites': SatellitesState,
267     'fixedpoints': FixedPointsState,
268     'observatories': ObservatoriesState,
269     'cadenceStructure': CadenceState,
270 }
271
272 def __init__(self, **kwargs):
273     if 'start_time' not in kwargs:
274         raise TypeError("SimulationState requires a 'start_time' parameter.")
275     start = kwargs['start_time']
276
277     defaults = {
278         'time': start,
279         'delta_time': 60.0,
280         'counts': None,
281         'pointing_spheres': {},
282         'detector': None,
283         'satellites': None,
284         'observatories': None,
285         'fixedpoints': None,
286         'celestial': None,
287         'cadenceStructure': None,
288     }
289     super().__init__(**{**defaults, **kwargs})
290     self.counts = CountsState(parent_sim=self)
291
292 def __setitem__(self, key, value):
293     if key == 'detector':
294         from detector import DetectorArray
295         cls = DetectorArray
296     else:
297         cls = self._FIELD_SCHEMAS.get(key)
298
299     if cls is not None and isinstance(value, dict) and not isinstance(value, cls):
300         value = cls(**value)
301     super().__setitem__(key, value)
302
303 def get_detector_positions(self, mask=None) -> np.ndarray:
304     """
305     Builds and returns an (N, 3) NumPy array of observer/detector positions
306     resolved by detector.category and detector.asset_index.
307
308     Args:
309         mask: Optional boolean or index array to subset the output.
310
311     Returns:
312         (N, 3) array of GCRS positions for detectors.
313     """
314     if self.detector is None or len(self.detector) == 0:
315         return np.zeros((0, 3), dtype=float)
316
317     det = self.detector
318     num_detectors = len(det)
319     category_array = np.array(det.category)
320     asset_index_array = det.asset_index
321
322     _pos_map = {}
323     if self.satellites and len(self.satellites) > 0:
324         _pos_map['satellites'] = self.satellites.position
325     if self.observatories and len(self.observatories) > 0:
326         _pos_map['observatories'] = self.observatories.position
327
328     all_positions = np.zeros((num_detectors, 3), dtype=float)
329     for i in range(num_detectors):
330         cat = category_array[i]
331         pos_array = _pos_map.get(cat)

```

```

332         if pos_array is not None:
333             all_positions[i] = pos_array[asset_index_array[i]]
334
335     if mask is not None:
336         return all_positions[mask]
337     return all_positions
338
339
340 def create_empty_simulation(start_time: datetime, delta_time: float = 60.0) ->
SimulationState:
341     """
342     Initializes a minimal, empty data structure for a space simulation.
343
344     Args:
345         start_time: The starting time and date of the simulation. This must be a
346                     timezone-aware datetime object set to UTC.
347         delta_time: The time step for the simulation in seconds.
348
349     Returns:
350         A SimulationState object representing the basic simulation state, not yet filled
351         except
352         for date and time and counts and pointing spheres.
353     """
354     if not isinstance(start_time, datetime):
355         raise TypeError("start_time must be a datetime object.")
356     if start_time.tzinfo is None:
357         raise ValueError("start_time must be timezone-aware. Please set tzinfo.")
358
359     return SimulationState(
360         start_time=start_time,
361         time=start_time,
362         delta_time=delta_time
363     )

```

Listing 2.1: minimalsimulation.py Source Code

Chapter 3

constants.py: Physical & Array Constants

Overview

Contains physical constants (Earth radius, AU, angular conversions) and element indices.

Literate Source Code

The source code of `constants.py` is documented below. Section label: [3](#).

```
1 # --- Global Constants for Array Indices ---
2 # These constants define column indices for numpy arrays, making the
3 # code more readable and preventing errors from using "magic numbers".
4 from math import pi
5
6
7 # -- Radii in Meters --
8 EARTH_RADIUS = 6378137.0
9 MOON_RADIUS = 1737400.0
10 GEO_RADIUS = 42164140.0
11 MOON_ORBIT_RADIUS = 384400000.0
12
13
14 # -- Some Useful Constants --
15 ARCSEC = 2.0 * pi / (360 * 3600)
16 DEGREE = 3600 * ARCSEC
17
18
19
20 # -- Orbital Elements Array Indices --
21 ORBITAL_A_IDX = 0           # Semi-major axis in meters
22 ORBITAL_E_IDX = 1           # Eccentricity (dimensionless)
23 ORBITAL_I_IDX = 2           # Inclination in radians
24 ORBITAL_RAAN_IDX = 3        # Right Ascension of the Ascending Node in radians
25 ORBITAL_ARGP_IDX = 4        # Argument of Perigee in radians
26 ORBITAL_M_IDX = 5           # Mean Anomaly in radians
27
28 # -- Pointing State Array Indices --
29 POINTING_COUNT_IDX = 0      # Number of points in the pointing grid
30 POINTING_PLACE_IDX = 1      # Current index in the pointing sequence
```

Listing 3.1: `constants.py` Source Code

Chapter 4

detector.py: Optoelectronic Detector Array

Overview

Defines DetectorArray (subclass of ComponentState) and detector construction/slicing functions.

Literate Source Code

The source code of detector.py is documented below. Section label: 4.

```
1 from typing import List, Any
2 from constants import *
3 import numpy as np
4 import math
5 from minimalsimulation import SchemaDict, ComponentState
6 from radiometry_data import FILTER_DATA
7 from radiometry_calcs import *
8
9
10 class DetectorArray(ComponentState):
11     """
12     DetectorArray models detector parameters for all satellites.
13
14     It inherits from SchemaDict so it supports both attribute-style
15     (dot notation) and dictionary subscript access. All array-valued fields
16     have a length equal to the number of detectors (satellites).
17
18     Fields:
19         apertureArea: Entrance-aperture area in m^2 (1-D array).
20         pixelOmega: Pixel solid angle in sr, assuming square pixels
21                     (1-D array).
22         qe: Total system quantum efficiency from entrance
23            aperture to detector (1-D array).
24         photoEff: Fraction of light captured in the photometry
25                  aperture (1-D array).
26         pixCount: Number of pixels in the detector array (1-D).
27         solarEx: Solar exclusion angle in radians (1-D array).
28         lunarEx: Lunar exclusion angle in radians (1-D array).
29         earthEx: Earth limb exclusion angle in radians (1-D array).
30         skyBack: Sky background flux in photons/s/m^2/sr (1-D).
31         zpCal: Filter zero-point calibration (1-D array).
32         integrationTime: Integration time in seconds (1-D array).
33         fov: Full field-of-view diameter in radians (1-D).
34         ifov: Pixel field-of-view in radians (1-D array).
35         filt: Filter band name for each detector (list of str).
36         category: The asset category this detector belongs to (e.g. 'satellites'
37                   or 'observatories') (list of str).
38         asset_index: The index of the asset within its category (1-D array of int).
39         pointing: Pointing unit vector for each detector,
40                  shape (n, 3).
41         pointing_state: Pointing scheduler state, shape (2, n).
```

```

41         Row 0 = total grid points, row 1 = current index.
42     """
43     def __init__(self, **kwargs):
44         defaults = {
45             'apertureArea': np.zeros(0, dtype=float),
46             'pixelOmega': np.zeros(0, dtype=float),
47             'qe': np.zeros(0, dtype=float),
48             'photoEff': np.zeros(0, dtype=float),
49             'pixCount': np.zeros(0, dtype=float),
50             'solarEx': np.zeros(0, dtype=float),
51             'lunarEx': np.zeros(0, dtype=float),
52             'earthEx': np.zeros(0, dtype=float),
53             'skyBack': np.zeros(0, dtype=float),
54             'zpCal': np.zeros(0, dtype=float),
55             'integrationTime': np.zeros(0, dtype=float),
56             'fov': np.zeros(0, dtype=float),
57             'ifov': np.zeros(0, dtype=float),
58             'filt': [],
59             'category': [],
60             'asset_index': np.zeros(0, dtype=int),
61             'pointing': np.zeros((0, 3), dtype=float),
62             'pointing_state': np.zeros((2, 0), dtype=int),
63         }
64         super().__init__(**{**defaults, **kwargs})
65
66
67     def setDetectorFOV(sim_data: Any, fovSize: float) -> None:
68         """
69         Sets the field-of-view of all detectors in the simulation to a single size.
70
71         This is an ad-hoc helper function mainly used for testing, not regular operation.
72
73         Args:
74             sim_data: The main simulation data structure.
75             fovSize: The new field-of-view diameter in radians.
76         """
77         det = sim_data.detector if hasattr(sim_data, 'detector') else sim_data['detector']
78         count = len(det.fov)
79         det.fov = np.full(count, fovSize)
80
81
82     def setDetectorIntegrationTime(sim_data: Any, itime: float) -> None:
83         """
84         Sets the integration time of all detectors in the simulation to a single value.
85
86         This is an ad-hoc helper function mainly used for testing, not regular operation.
87
88         Args:
89             sim_data: The main simulation data structure.
90             itime: The new integration time in seconds.
91         """
92         det = sim_data.detector if hasattr(sim_data, 'detector') else sim_data['detector']
93         count = len(det.fov)
94         det.integrationTime = np.full(count, itime)
95
96
97     #####
98
99     def makeBlankDetector(n: int) -> DetectorArray:
100         """
101         makeBlankDetector makes and returns a DetectorArray
102         with parameters initialized to zero arrays of length n.
103         """
104         return DetectorArray(
105             apertureArea=np.zeros(n, dtype=float),
106             pixelOmega=np.zeros(n, dtype=float),
107             qe=np.zeros(n, dtype=float),
108             photoEff=np.zeros(n, dtype=float),
109             pixCount=np.zeros(n, dtype=float),
110             solarEx=np.zeros(n, dtype=float),
111             lunarEx=np.zeros(n, dtype=float),
112             earthEx=np.zeros(n, dtype=float),
113             skyBack=np.zeros(n, dtype=float),

```

```

114     zpCal=np.zeros(n, dtype=float),
115     integrationTime=np.zeros(n, dtype=float),
116     fov=np.zeros(n, dtype=float),
117     ifov=np.zeros(n, dtype=float),
118     filt=[""] * n,
119     category=[""] * n,
120     asset_index=np.zeros(n, dtype=int),
121     pointing=np.zeros((n, 3), dtype=float),
122     pointing_state=np.zeros((2, n), dtype=int)
123 )
124
125 #####
126
127 def makeDetector(n, band, fov, ifov, aper, intTime: float = 1.0,
128                 qe=0.5, photfrac=0.7,
129                 solarex=20.0 * DEGREE, lunarex=10.0 * DEGREE,
130                 earthex=15.0 * DEGREE, category=None, asset_index=None):
131     '''
132     makeDetector takes parameters of a sensor and stuffs a filter array
133     and a detector array, which it returns.
134
135     Args:
136         n:          Number of sensors to produce.
137         band:       Band the measurement takes place in (see radiometry_data).
138         fov:       Full field-of-view diameter in radians.
139         ifov:      Pixel field-of-view diameter in radians.
140         aper:      Aperture diameter - assumed to be a disk - in meters.
141                   The actual value stored and used is the apertureArea.
142         intTime:   Integration time in seconds. Defaults to 1.0.
143         qe:        Quantum efficiency of the system from entrance aperture
144                   to detector.
145         photfrac:  Fraction of light captured in the photometry aperture.
146         solarex:   Solar exclusion angle in radians.
147         lunarex:   Lunar exclusion angle in radians.
148         earthex:   Earth limb exclusion angle in radians.
149
150     Creates a detector for any platform (space or ground).
151     '''
152     detect = makeBlankDetector(n)
153     detect.apertureArea[:] = math.pi * (aper / 2) ** 2
154     detect.pixelOmega[:] = ifov ** 2
155     detect.qe[:] = qe
156     detect.photoEff[:] = photfrac
157     pixels = (fov / ifov) ** 2
158     detect.pixCount[:] = pixels
159     detect.solarEx[:] = solarex
160     detect.lunarEx[:] = lunarex
161     detect.earthEx[:] = earthex
162     detect.skyBack[:] = (
163         amag(FILTER_DATA[band]['sky'])
164         * FILTER_DATA[band]['zero_point']
165         / (ARCSEC ** 2)
166     )
167     detect.zpCal[:] = FILTER_DATA[band]['zero_point']
168     detect.integrationTime[:] = intTime
169     detect.fov[:] = fov
170     detect.ifov[:] = ifov
171     detect.filt = [band] * n
172     detect.category = category if category is not None else [""] * n
173     detect.asset_index = asset_index if asset_index is not None else np.zeros(n, dtype=
174 int)
175     detect.pointing = np.zeros((n, 3), dtype=float)
176     detect.pointing_state = np.zeros((2, n), dtype=int)
177     return detect
178
179 def requiredIntegrationTime(limitingMag, SNR, d, debug=0):
180     '''
181     Calculates the required integration time to achieve a given limiting
182     magnitude with a specified signal-to-noise ratio (SNR).
183
184     Args:
185         limitingMag (float): The desired limiting magnitude.

```

```

186     SNR (float):          The target signal-to-noise ratio.
187     d (DetectorArray):   A detector object containing the required
188                           parameters.
189     debug (int, optional): If set to 1, prints the intermediate
190                           variables used in the calculation.
191                           Defaults to 0.
192
193     This function is based on the radiometric equation, solving for
194     the integration time t:
195     SNR = (Signal) / sqrt(Signal + Background)
196     t = SNR^2 * (beta * omega) / (alpha^2 * A * eta * f^2)
197
198     Returns:
199         float: The required integration time in seconds.
200     '''
201     gamma = SNR
202     beta = d.skyBack
203     omega = d.pixelOmega
204     alpha = amag(limitingMag) * d.zpCal
205     A = d.apertureArea
206     eta = d.qe
207     f = d.photoEff
208     if debug == 1:
209         print(f"gamma {gamma:.2e}")
210         print("beta is,", beta)
211         print("omega", omega)
212         print("alpha is", alpha)
213         print('A', A)
214         print('eta', eta)
215         print('f', f)
216     t = gamma ** 2 * beta * omega / (alpha ** 2 * A * eta * (f ** 2))
217     return t
218
219
220 def appendDetector(cd: DetectorArray, new_cd: DetectorArray) -> None:
221     """
222     Appends the attributes of new_cd to the existing detector object cd
223     in-place.
224
225     Iterates over all declared DetectorArray dataclass fields.  Numpy
226     1-D arrays are concatenated with np.append; 2-D arrays are joined
227     with np.vstack (or np.hstack for pointing_state which is (2, n));
228     list fields are extended.
229     """
230     for attr_name in cd.keys():
231         new_val = getattr(new_cd, attr_name)
232         cur_val = getattr(cd, attr_name)
233         if isinstance(cur_val, np.ndarray):
234             if cur_val.ndim == 1:
235                 setattr(cd, attr_name, np.append(cur_val, new_val))
236             elif cur_val.ndim == 2:
237                 if attr_name == 'pointing_state':
238                     setattr(cd, attr_name, np.hstack([cur_val, new_val]))
239                 else:
240                     setattr(cd, attr_name, np.vstack([cur_val, new_val]))
241         elif isinstance(cur_val, list):
242             cur_val.extend(new_val)
243
244
245 #####
246
247 def testdetector():
248     '''
249     testdetector creates an example that can be compared
250     with some of the stuff in Curio.
251     '''
252     detect = makeDetector(1, "V", 1 * DEGREE,
253                           2 * ARCSEC, 1,
254                           qe=0.2, photfrac=1.0)
255     print(requiredIntegrationTime(20.5, 10, detect, debug=1))
256
257 if __name__ == '__main__':

```

Listing 4.1: detector.py Source Code

Chapter 5

propagation.py: Satellite Orbit Propagation

Overview

Implements SGP4 TLE loading and Keplerian orbit propagation for satellites.

Literate Source Code

The source code of `propagation.py` is documented below. Section label: 5.

```
1 import numpy as np
2 from datetime import datetime, timezone
3 from typing import Dict, Any, List, Tuple
4 from sgp4.api import Satrec
5 from astropy.time import Time
6 from astropy.coordinates import get_body, GCRS
7 import astropy.units as u
8
9 from constants import *
10
11
12
13 def add_satellites_from_tle(sim_data: Any, tle_file_path: str, sat_category: str) ->
14     None:
15     """
16     Adds and initializes a category of satellites from a TLE file.
17     Note although the TLEs are loaded, positions etc. are not.
18
19     Args:
20         sim_data: The main simulation data structure.
21         tle_file_path: Path to the TLE file.
22         sat_category: The key for this satellite category (e.g., 'satellites').
23
24     Data added to the set_category element of sim_data
25     position
26     velocity
27     acceleration
28     orbital_elements
29     epochs
30     """
31     from detector import makeBlankDetector, appendDetector
32     orbital_elements, epochs = readtle(tle_file_path)
33     num_sats = len(epochs)
34
35     sim_data.counts[sat_category] = num_sats
36     new_detector = makeBlankDetector(num_sats)
37     new_detector.category = [sat_category] * num_sats
38     new_detector.asset_index = np.arange(num_sats, dtype=int)
39     if 'detector' not in sim_data or not sim_data.detector:
40         sim_data.detector = new_detector
41     else:
```

```

41     appendDetector(sim_data.detector, new_detector)
42     sim_data[sat_category] = {
43         'position': np.zeros((num_sats, 3), dtype=float),
44         'velocity': np.zeros((num_sats, 3), dtype=float),
45         'acceleration': np.zeros((num_sats, 3), dtype=float),
46         'orbital_elements': orbital_elements,
47         'epochs': epochs,
48     }
49
50 def readtle(tle_file_path: str) -> Tuple[np.ndarray, List[datetime]]:
51     """
52     Reads a TLE file and extracts orbital elements and epochs for each satellite.
53
54     The array returned has the orbital elements in "canonical" order.
55     """
56     orbital_elements_list = []
57     epochs_list = []
58     with open(tle_file_path, 'r') as f:
59         lines = f.readlines()
60
61     for i in range(0, len(lines), 3):
62         line1 = lines[i+1].strip()
63         line2 = lines[i+2].strip()
64
65         satellite = Satrec.twoline2rv(line1, line2)
66
67         jd, fr = satellite.jdsatepoch, satellite.jdsatepochF
68         epoch_dt = Time(jd, fr, format='jd', scale='utc').to_datetime(timezone.utc)
69         epochs_list.append(epoch_dt)
70
71         a = satellite.a * satellite.radiusearthkm * 1000.0
72         e = satellite.ecco
73         inc = satellite.inclo
74         raan = satellite.nodeo
75         argp = satellite.argpo
76         M = satellite.mo
77
78         elements = np.zeros(6)
79         elements[ORBITAL_A_IDX] = a
80         elements[ORBITAL_E_IDX] = e
81         elements[ORBITAL_I_IDX] = inc
82         elements[ORBITAL_RAAN_IDX] = raan
83         elements[ORBITAL_ARGP_IDX] = argp
84         elements[ORBITAL_M_IDX] = M
85         orbital_elements_list.append(elements)
86
87     return np.array(orbital_elements_list, dtype=float), epochs_list
88
89 def propagate_satellites(data_struct: Any, time_date: datetime, sat_category: str = None) -> Any:
90     """
91     data_struct is the "standard" structure for the simulation
92     time_date - a datetime - is the time that the satellites are propagated to.
93     NOTE that the datetime structure in simulation_data is NOT updated to this value by
94     this function!
95
96     Updates satellite positions and pointing vectors based on their orbital elements.
97
98     """
99     MU_EARTH = 3.986004418e14
100     time_date_timestamp = time_date.timestamp()
101
102     if sat_category:
103         categories = [sat_category]
104     elif hasattr(data_struct, 'counts') and data_struct.counts:
105         non_sat_keys = {'celestial', 'observatories', 'fixedpoints'}
106         categories = [
107             k for k, v in data_struct.counts.items()
108             if k not in non_sat_keys and v > 0
109         ]
110     if not categories:
111         categories = ['satellites']
112     else:

```

```

112     categories = ['satellites']
113
114     for category in categories:
115         if category not in data_struct.counts or data_struct.counts[category] == 0:
116             continue
117
118         elements = data_struct[category].orbital_elements
119         epochs = data_struct[category].epochs
120
121         epoch_timestamps = np.array([e.timestamp() for e in epochs])
122         delta_t_array = time_date_timestamp - epoch_timestamps
123
124         a = elements[:, ORBITAL_A_IDX]
125         e = elements[:, ORBITAL_E_IDX]
126         i = elements[:, ORBITAL_I_IDX]
127         raan = elements[:, ORBITAL_RAAN_IDX]
128         argp = elements[:, ORBITAL_ARGP_IDX]
129         M0 = elements[:, ORBITAL_M_IDX]
130
131         # Guard against uninitialized/invalid orbits (a <= 0 or e >= 1.0)
132         safe_a = np.where(a <= 0, EARTH_RADIUS + 400000.0, a)
133         safe_e = np.clip(e, 0.0, 0.999999)
134
135         n = np.sqrt(MU_EARTH / safe_a**3)
136         M = (M0 + n * delta_t_array) % (2 * np.pi)
137
138         E = M.copy()
139         for _ in range(10):
140             f_E = E - safe_e * np.sin(E) - M
141             f_prime_E = 1 - safe_e * np.cos(E)
142             f_prime_E[f_prime_E == 0] = 1e-10
143             E = E - f_E / f_prime_E
144
145         tan_nu_half = np.sqrt((1 + safe_e) / (1 - safe_e)) * np.tan(E / 2)
146         nu = 2 * np.arctan(tan_nu_half)
147
148         r = safe_a * (1 - safe_e * np.cos(E))
149
150         x_pqw = r * np.cos(nu)
151         y_pqw = r * np.sin(nu)
152
153         cos_raan = np.cos(raan)
154         sin_raan = np.sin(raan)
155         cos_argp = np.cos(argp)
156         sin_argp = np.sin(argp)
157         cos_i = np.cos(i)
158         sin_i = np.sin(i)
159
160         P_x = cos_argp * cos_raan - sin_argp * sin_raan * cos_i
161         P_y = cos_argp * sin_raan + sin_argp * cos_raan * cos_i
162         P_z = sin_argp * sin_i
163
164         Q_x = -sin_argp * cos_raan - cos_argp * sin_raan * cos_i
165         Q_y = -sin_argp * sin_raan + cos_argp * cos_raan * cos_i
166         Q_z = cos_argp * sin_i
167
168         x_gcrs = x_pqw * P_x + y_pqw * Q_x
169         y_gcrs = x_pqw * P_y + y_pqw * Q_y
170         z_gcrs = x_pqw * P_z + y_pqw * Q_z
171
172         positions = np.vstack((x_gcrs, y_gcrs, z_gcrs)).T
173         data_struct[category].position = positions
174
175         norms = np.linalg.norm(positions, axis=1)[: , np.newaxis]
176         norms[norms == 0] = 1.0
177
178
179     return data_struct

```

Listing 5.1: propagation.py Source Code

Chapter 6

observatories.py: Ground Station Observatories

Overview

Implements GCRS coordinate conversion and sidereal velocity calculations for ground stations.

Literate Source Code

The source code of `observatories.py` is documented below. Section label: 6.

```
1 import numpy as np
2 from typing import Dict, Any
3
4 def add_observatories(
5     sim_data: Any,
6     num_observatories: int,
7     latitudes: np.ndarray = None,
8     longitudes: np.ndarray = None,
9     altitudes: np.ndarray = None
10 ) -> None:
11     """
12     Adds observatory data structures to the simulation data.
13
14     Args:
15         sim_data: The main simulation data structure.
16         num_observatories: The number of observatories to add.
17         latitudes: Latitude coordinates in degrees.
18         longitudes: Longitude coordinates in degrees.
19         altitudes: Altitude coordinates in meters.
20     """
21     if not isinstance(num_observatories, int) or num_observatories < 0:
22         raise ValueError("num_observatories must be a non-negative integer.")
23
24     if latitudes is None:
25         latitudes = np.zeros(num_observatories)
26     if longitudes is None:
27         longitudes = np.zeros(num_observatories)
28     if altitudes is None:
29         altitudes = np.zeros(num_observatories)
30
31     from detector import makeBlankDetector, appendDetector
32     detector = makeBlankDetector(num_observatories)
33     detector.category = ['observatories'] * num_observatories
34
35     from minimalsimulation import ObservatoriesState
36     if sim_data.counts.get('observatories', 0) > 0:
37         # Append to existing
38         existing = sim_data.observatories
39         detector.asset_index = np.arange(sim_data.counts.observatories, sim_data.counts.
40 observatories + num_observatories, dtype=int)
41         sim_data.counts.observatories += num_observatories
```

```

41     existing.latitude = np.append(existing.latitude, np.array(latitudes, dtype=float
42 ))
43     existing.longitude = np.append(existing.longitude, np.array(longitudes, dtype=
44 float))
45     existing.altitude = np.append(existing.altitude, np.array(altitudes, dtype=float
46 ))
47     existing.position = np.vstack([existing.position, np.zeros((num_observatories,
48 3), dtype=float)])
49     existing.velocity = np.vstack([existing.velocity, np.zeros((num_observatories,
50 3), dtype=float)])
51     existing.acceleration = np.vstack([existing.acceleration, np.zeros((
52 num_observatories, 3), dtype=float)])
53     existing.pointing = np.vstack([existing.pointing, np.zeros((num_observatories,
54 3), dtype=float)])
55 else:
56     sim_data.counts.observatories = num_observatories
57     detector.asset_index = np.arange(num_observatories, dtype=int)
58     sim_data.observatories = ObservatoriesState(
59         latitude=np.array(latitudes, dtype=float),
60         longitude=np.array(longitudes, dtype=float),
61         altitude=np.array(altitudes, dtype=float),
62         position=np.zeros((num_observatories, 3), dtype=float),
63         velocity=np.zeros((num_observatories, 3), dtype=float),
64         acceleration=np.zeros((num_observatories, 3), dtype=float),
65         pointing=np.zeros((num_observatories, 3), dtype=float),
66     )
67
68     if 'detector' not in sim_data or not sim_data.detector:
69         sim_data.detector = detector
70     else:
71         appendDetector(sim_data.detector, detector)
72
73     # Initial propagation of observatory positions to the current simulation time
74     propagate_observatories(sim_data, sim_data.time)
75
76 def propagate_observatories(sim_data: Any, time_date: Any) -> None:
77     """
78     Propagates observatory positions in GCRS (ECI) based on Earth rotation.
79     """
80     obs = sim_data.observatories
81     if not obs or len(obs.latitude) == 0:
82         return
83
84     from astropy.coordinates import EarthLocation
85     from astropy.time import Time
86     import astropy.units as u
87
88     # Define locations
89     locations = EarthLocation(
90         lat=obs.latitude * u.deg,
91         lon=obs.longitude * u.deg,
92         height=obs.altitude * u.m
93     )
94
95     t = Time(time_date)
96     # Get coordinates in Geocentric Celestial Reference System (GCRS)
97     gcrs_coords = locations.get_gcrs(t)
98
99     # GCRS coordinates are in meters
100     x = gcrs_coords.cartesian.x.to(u.m).value
101     y = gcrs_coords.cartesian.y.to(u.m).value
102     z = gcrs_coords.cartesian.z.to(u.m).value
103
104     # If x, y, z are scalars (e.g. single observatory), stack them correctly
105     if np.isscalar(x):
106         obs.position = np.array([[x, y, z]], dtype=float)
107     else:
108         obs.position = np.vstack([x, y, z]).T
109
110     # Compute velocity via finite-difference of GCRS positions.
111     # The naive omegaxr formula is only valid in the ECEF/ITRS

```

```

107 # (body-fixed) frame, but obs.position is in GCRS (inertial).
108 # A small Deltat finite-difference gives the correct GCRS velocity
109 # including precession and nutation effects.
110 from datetime import timedelta
111 dt_seconds = 1.0
112 t2 = Time(time_date + timedelta(seconds=dt_seconds))
113 gcrs_coords2 = locations.get_gcrs(t2)
114
115 x2 = gcrs_coords2.cartesian.x.to(u.m).value
116 y2 = gcrs_coords2.cartesian.y.to(u.m).value
117 z2 = gcrs_coords2.cartesian.z.to(u.m).value
118
119 if np.isscalar(x2):
120     pos2 = np.array([[x2, y2, z2]], dtype=float)
121 else:
122     pos2 = np.vstack([x2, y2, z2]).T
123
124 obs.velocity = (pos2 - obs.position) / dt_seconds

```

Listing 6.1: observatories.py Source Code

Chapter 7

celestialbodies.py: Celestial Bodies Ephemeris

Overview

Updates Sun and Moon positions in GCRS coordinates over simulation time.

Literate Source Code

The source code of `celestialbodies.py` is documented below. Section label: 7.

```
1 import numpy as np
2 from typing import Dict, Any, Optional
3 from datetime import datetime
4 from astropy.time import Time
5 from astropy.coordinates import get_body, GCRS
6 import astropy.units as u
7
8 def add_celestial_bodies(sim_data: Any) -> None:
9     """
10     Adds celestial body structures (for Sun and Moon) to the simulation data.
11
12     Args:
13         sim_data: The simulation data structure.
14     """
15     sim_data.counts.celestial = 2 # Sun and Moon
16     sim_data.celestial = {
17         'position': np.zeros((2, 3), dtype=float),
18         'velocity': np.zeros((2, 3), dtype=float),
19         'acceleration': np.zeros((2, 3), dtype=float),
20     }
21
22 def celestial_update(data_struct: Any, time_date: Optional[datetime] = None) -> Any:
23     """
24     Calculates and updates the positions of the Sun and Moon.
25     """
26     if time_date is None:
27         time_date = data_struct.time
28
29     if time_date.tzinfo is None:
30         raise ValueError("time_date must be timezone-aware.")
31
32     astro_time = Time(time_date)
33     sun_coords = get_body("sun", astro_time)
34     sun_gcrs = sun_coords.transform_to(GCRS(obstime=astro_time))
35     moon_coords = get_body("moon", astro_time)
36     moon_gcrs = moon_coords.transform_to(GCRS(obstime=astro_time))
37
38     celestial_pos = data_struct.celestial.position
39     celestial_pos[0] = sun_gcrs.cartesian.xyz.to(u.m).value
40     celestial_pos[1] = moon_gcrs.cartesian.xyz.to(u.m).value
41
```

42

```
return data_struct
```

Listing 7.1: celestialbodies.py Source Code

Chapter 8

targets.py: Reference Target Generation

Overview

Generates logarithmic spherical distributions of fixed reference targets.

Literate Source Code

The source code of `targets.py` is documented below. Section label: 8.

```
1 import numpy as np
2 # from datetime import datetime, timezone
3 from typing import Dict, Any
4 from generate_log_spherical_points import generate_log_spherical_points
5 from constants import *
6 from minimalsimulation import FixedPointsState
7
8 def add_fixed_points(
9     sim_data: Any,
10     num_points: int = 100,
11     size: float = 1.0,
12     innerRadius: float = 2000000,
13     outerRadius: float = 2 * GEO_RADIUS
14 ) -> None:
15     """
16     Adds a structure for fixed reference points in the GCRS frame.
17
18     Args:
19         sim_data: The simulation data structure.
20         num_points: The number of fixed points to generate.
21         size: the size of the objects,
22         innerRadius: The minimum radius at which test points will be created.
23         outerRadius: The maximum radius at which test points will be created.
24     """
25     positions = generate_log_spherical_points(
26         num_points=num_points,
27         inner_radius=innerRadius,
28         outer_radius=outerRadius
29     )
30     if not sim_data.fixedpoints:
31         sim_data.fixedpoints = FixedPointsState()
32     sim_data.fixedpoints.add_target(positions, size=size, albedo=0.2)
```

Listing 8.1: targets.py Source Code

Chapter 9

exclusion.py: Celestial Obstruction & Exclusion Zones

Overview

Computes solar, lunar, and Earth limb/horizon exclusion zones for space and ground sensors.

Literate Source Code

The source code of `exclusion.py` is documented below. Section label: 9.

```
1 import numpy as np
2 from typing import Dict, Any, Tuple, Optional
3
4 from constants import (
5     EARTH_RADIUS, MOON_RADIUS
6 )
7 def exclusion(
8     data_struct: Dict[str, Any],
9     satellite_index: int,
10    sat_category: str = 'satellites',
11    print_debug: bool = False
12 ) -> int:
13     """
14     Determines if a satellite's pointing vector is excluded by the Sun, Moon, or Earth.
15
16     This function is vectorized to check for exclusions from all three bodies (Sun,
17     Moon, Earth) simultaneously for a single satellite.
18
19     Args:
20         data_struct: The main simulation data dictionary.
21         satellite_index: The index of the satellite to check.
22         sat_category: The satellite category (e.g. 'satellites', 'red_satellites').
23         print_debug: If True, prints detailed debug information for the calculation.
24
25     Returns:
26         1 if the satellite's view is excluded by any of the bodies, 0 otherwise.
27     """
28     # Determine asset category and index using the detector's internal tracking
29     detector_props = data_struct.detector
30     asset_category = detector_props.category[satellite_index]
31     asset_idx = detector_props.asset_index[satellite_index]
32
33     sat_pos = getattr(data_struct, asset_category).position[asset_idx]
34     sat_pointing = data_struct.detector.pointing[satellite_index]
35
36     norm_pointing = np.linalg.norm(sat_pointing)
37     if norm_pointing < 1e-9:
38         return 1 # Not pointing anywhere, treat as excluded
39
40     u_sat_pointing = sat_pointing / norm_pointing
41
```

```

42 detector_props = data_struct.detector
43
44 # Local horizon exclusion check for ground-based observatories
45 if asset_category == 'observatories':
46     norm_pos = np.linalg.norm(sat_pos)
47     if norm_pos > 1e-9:
48         zenith_normal = sat_pos / norm_pos
49         cos_zenith = np.clip(np.dot(zenith_normal, u_sat_pointing), -1.0, 1.0)
50         zenith_angle = np.arccos(cos_zenith)
51         # Minimum elevation angle is stored in earthEx
52         min_elevation = detector_props.earthEx[satellite_index]
53         max_zenith = np.pi / 2.0 - min_elevation
54         if zenith_angle > max_zenith:
55             if print_debug:
56                 print(f"--- Horizon Exclusion for Observatory {asset_idx} ---")
57                 print(f"    Zenith Angle: {np.rad2deg(zenith_angle):.2f} deg, "
58                       f"    Max Zenith: {np.rad2deg(max_zenith):.2f} deg (elev < {np.
59 rad2deg(min_elevation):.2f} deg)")
60             return 1
61
62 # Celestial body positions and radii
63 body_positions = np.array([
64     data_struct.celestial.position[0], # Sun
65     data_struct.celestial.position[1], # Moon
66     [0.0, 0.0, 0.0] # Earth's center
67 ])
68 body_radii = np.array([0.0, MOON_RADIUS, EARTH_RADIUS])
69
70 # Satellite-specific exclusion angles
71 exclusion_angles = np.array([
72     detector_props.solarEx[satellite_index],
73     detector_props.lunarEx[satellite_index],
74     detector_props.earthEx[satellite_index]
75 ])
76
77 # --- Vectorized Calculations ---
78 vecs_to_bodies = body_positions - sat_pos
79 dists_to_bodies = np.linalg.norm(vecs_to_bodies, axis=1)
80
81 # Calculate angles to bodies, handling potential division by zero
82 angles = np.full(3, np.pi)
83 valid_dists = dists_to_bodies > 1e-9
84
85 u_vecs_to_bodies = np.zeros_like(vecs_to_bodies)
86 u_vecs_to_bodies[valid_dists] = vecs_to_bodies[valid_dists] / dists_to_bodies[
87 valid_dists, np.newaxis]
88
89 cos_angles = np.clip(np.dot(u_vecs_to_bodies, u_sat_pointing), -1.0, 1.0)
90 angles = np.arccos(cos_angles)
91
92 # Calculate apparent radii (for Moon and Earth)
93 apparent_radii = np.zeros(3)
94 non_zero_dists = (dists_to_bodies[1:] > 0)
95 apparent_radii[1:][non_zero_dists] = np.arctan(body_radii[1:][non_zero_dists] /
96 dists_to_bodies[1:][non_zero_dists])
97
98 # Check for exclusion
99 is_excluded = (angles - apparent_radii) < exclusion_angles
100
101 if asset_category == 'observatories':
102     # For ground stations, Earth limb checks are replaced by the horizon check above
103     is_excluded[2] = False
104
105 if print_debug:
106     body_names = ["Sun", "Moon", "Earth"]
107     asset_name = "Observatory" if asset_category == "observatories" else "Satellite"
108     print(f"--- Exclusion Debug for {asset_name} {asset_idx} ---")
109     for i in range(3):
110         print(f"    - {body_names[i]:<5} Flag: {is_excluded[i]}, "
111               f"    Angle: {np.rad2deg(angles[i]):.2f} deg, "
112               f"    Excl: {np.rad2deg(exclusion_angles[i]):.2f} deg")

```

```

111     return 1 if np.any(is_excluded) else 0
112
113 def update_exclusion_table(
114     data_struct: Dict[str, Any],
115     print_debug_for_sat: Optional[int] = None
116 ) -> None:
117     """
118     Updates the exclusion table for all detectors against all
119     fixed points. Uses the detector category / asset_index
120     mapping so that satellites and observatories are handled
121     uniformly.
122
123     For space-based detectors the standard Sun / Moon / Earth-limb
124     exclusion checks are applied. For ground-based observatories
125     the Earth-limb check is replaced by a local-horizon elevation
126     check (the minimum elevation is stored in detector.earthEx).
127
128     The resulting matrix is stored in
129     data_struct.fixedpoints.exclusion with shape
130     (num_fixed_points, num_detectors).
131     """
132     num_fixed_points = data_struct.counts.get('fixedpoints', 0)
133     num_detectors = len(data_struct.detector.filt)
134
135     if num_detectors == 0 or num_fixed_points == 0:
136         return
137
138     targets = data_struct.fixedpoints.position
139     detector_props = data_struct.detector
140
141     # --- Build observer positions using get_detector_positions ---
142     category_array = np.array(detector_props.category)
143     observer_pos = data_struct.get_detector_positions()
144
145     # --- Pointing vectors: (num_targets, num_detectors, 3) ---
146     pointing_vectors = (
147         targets[:, np.newaxis, :]
148         - observer_pos[np.newaxis, :, :]
149     )
150
151     norm_pointing = np.linalg.norm(pointing_vectors, axis=2)
152     safe_norm = np.where(norm_pointing == 0, 1.0, norm_pointing)
153     u_pointing = pointing_vectors / safe_norm[:, :, np.newaxis]
154
155     # --- Celestial body geometry ---
156     body_positions = np.array([
157         data_struct.celestial.position[0],
158         data_struct.celestial.position[1],
159         [0.0, 0.0, 0.0]
160     ])
161     body_radii = np.array([0.0, MOON_RADIUS, EARTH_RADIUS])
162
163     # vecs_to_bodies: (num_detectors, 3_bodies, 3_xyz)
164     vecs_to_bodies = (
165         body_positions[np.newaxis, :, :]
166         - observer_pos[:, np.newaxis, :]
167     )
168     dists_to_bodies = np.linalg.norm(vecs_to_bodies, axis=2)
169     safe_dists = np.where(
170         dists_to_bodies == 0, 1.0, dists_to_bodies
171     )
172     u_vecs_to_bodies = (
173         vecs_to_bodies / safe_dists[:, :, np.newaxis]
174     )
175
176     # Angles: (num_targets, num_detectors, 3_bodies)
177     # t = target, d = detector, b = body, i = xyz
178     cos_angles = np.einsum(
179         'tdi,dbi->tdb', u_pointing, u_vecs_to_bodies
180     )
181     cos_angles = np.clip(cos_angles, -1.0, 1.0)
182     angles = np.arccos(cos_angles)
183

```

```

184 # Apparent angular radii: (num_detectors, 3_bodies)
185 apparent_radii = np.zeros((num_detectors, 3))
186 apparent_radii[:, 1:] = np.arctan(
187     body_radii[1:][np.newaxis, :] / safe_dists[:, 1:]
188 )
189
190 # Exclusion angles: (num_detectors, 3_bodies)
191 exclusion_angles = np.vstack([
192     detector_props.solarEx,
193     detector_props.lunarEx,
194     detector_props.earthEx
195 ]).T
196
197 # Exclusion test: (num_targets, num_detectors, 3_bodies)
198 is_excluded = (
199     (angles - apparent_radii[np.newaxis, :, :])
200     < exclusion_angles[np.newaxis, :, :]
201 )
202
203 # --- Observatory horizon check (replaces Earth-limb) ---
204 obs_mask = (category_array == 'observatories')
205 if np.any(obs_mask):
206     obs_indices = np.where(obs_mask)[0]
207     obs_positions = observer_pos[obs_indices]
208     obs_norms = np.linalg.norm(obs_positions, axis=1)
209     safe_obs_norms = np.where(
210         obs_norms == 0, 1.0, obs_norms
211     )
212     zenith_vecs = (
213         obs_positions / safe_obs_norms[:, np.newaxis]
214     )
215
216 # cos(zenith angle) for every (target, obs) pair
217 cos_zenith = np.einsum(
218     'toi,oi->to',
219     u_pointing[:, obs_indices, :],
220     zenith_vecs
221 )
222 cos_zenith = np.clip(cos_zenith, -1.0, 1.0)
223 zenith_angles = np.arccos(cos_zenith)
224
225 min_elev = exclusion_angles[obs_indices, 2]
226 max_zenith = np.pi / 2.0 - min_elev
227
228 # Replace Earth-body exclusion with horizon check
229 is_excluded[:, obs_indices, 2] = (
230     zenith_angles > max_zenith[np.newaxis, :]
231 )
232
233 # Collapse body axis -> scalar excluded flag
234 exclusion_matrix = np.any(is_excluded, axis=2).astype(int)
235
236 # Zero-norm pointing -> treat as excluded
237 exclusion_matrix[norm_pointing < 1e-9] = 1
238
239 # --- Debug output ---
240 if (print_debug_for_sat is not None
241     and 0 <= print_debug_for_sat < num_detectors):
242     original_pointing = data_struct.detector.pointing.copy()
243     for j in range(num_fixed_points):
244         data_struct.detector.pointing[print_debug_for_sat] = (
245             pointing_vectors[j, print_debug_for_sat]
246         )
247     exclusion(
248         data_struct,
249         print_debug_for_sat,
250         print_debug=True,
251     )
252     data_struct.detector.pointing = original_pointing
253
254 data_struct.fixedpoints.exclusion = exclusion_matrix

```

Listing 9.1: exclusion.py Source Code

Chapter 10

pointing.py: Sensor Pointing & Re-slewing

Overview

Generates spherical pointing grids, updates boresight vectors, and executes random perturbation (jerk).

Literate Source Code

The source code of `pointing.py` is documented below. Section label: 10.

```
1 import numpy as np
2 from typing import Dict, Any
3 import math
4 from datetime import datetime, timezone
5 import plotly.graph_objects as go
6 import os
7
8 from minimalsimulation import create_empty_simulation
9 from celestialbodies import add_celestial_bodies, celestial_update
10 from propagation import add_satellites_from_tle, propagate_satellites
11 from plotting_3d import plot_3d_scatter
12 from constants import POINTING_COUNT_IDX, POINTING_PLACE_IDX
13 from fibonacciSearch import pointing_vectors, resort_vectors_by_proximity
14 from exclusion import exclusion
15
16
17 def generate_pointing_sphere(sim_data: Any, n_points: int, debug: bool = False) -> None:
18     """
19     Generates a pointing sphere with n_points and stores it in the sim_data.
20     pointing_spheres[n_points]
21     A pointing sphere is a 3 by n_points numpy array with the 3 representing unit
22     vectors to be
23     pointed to
24     These positions will be used by the update_detector_pointing to
25     point sensors incrementally.
26     The index is sim_data.pointing_spheres[n]
27     If a sphere with the same number of points already exists, this function does
28     nothing.
29     The current version of the code resorts the vector by proximity to make the sky
30     search
31     more efficient, although this is not deeply optimized yet.
32     """
33     if n_points not in sim_data.pointing_spheres:
34         print(f"Generating pointing sphere with {n_points} points...")
35         sim_data.pointing_spheres[n_points] = \
36             resort_vectors_by_proximity(pointing_vectors(n_points))
37
38     if debug:
39         print(f"\n--- Debugging Pointing Sphere (n_points={n_points}) ---")
40         generated_vectors = sim_data.pointing_spheres[n_points]
41         print(f"Total number of pointing vectors generated: {len(generated_vectors)}")
```

```

38
39     num_to_show = min(5, len(generated_vectors))
40     print(f"Showing first {num_to_show} pointing vectors and their norms:")
41     print(f"{'Index':<7} {'Vector (x, y, z)':<35} {'Norm':<10}")
42     print(f"{'-----':<7} {'-----':<35} {'-----':<10}")
43     for i in range(num_to_show):
44         vec = generated_vectors[i]
45         norm = np.linalg.norm(vec)
46         print(f"{i:<7} {str(vec):<35} {norm:<10.6f}")
47     print("-----\n")
48
49
50 def update_detector_pointing(sim_data: Any, sat_category: str = 'satellites', debug:
51     bool = False) -> None:
52     """
53     Updates the pointing vector for each detector, skipping excluded pointing directions
54     """
55     num_detectors = len(sim_data.detector.filt)
56     if num_detectors == 0:
57         return
58
59     # Bring in the appropriate pieces of the data structure for easier reference
60     pointing_state = sim_data.detector.pointing_state
61     # print('pointing_state in pointing.py ', pointing_state)
62     pointing_vectors_all = sim_data.detector.pointing
63
64     # Iterate over satellites
65     for i in range(num_detectors):
66         # Place a grid of vectors to use in grid
67         count = int(pointing_state[POINTING_COUNT_IDX, i])
68         if count == 0:
69             continue
70         grid = sim_data.pointing_spheres[count]
71
72         place = int(pointing_state[POINTING_PLACE_IDX, i])
73         start_place = place
74
75         while True:
76             # Move to the next place, wrap around if at end
77             place += 1
78             if place >= count:
79                 place = 0
80             pointing_vectors_all[i] = grid[place]
81
82             excluded = exclusion(sim_data, i, sat_category=sat_category)
83             if debug:
84                 print(f"Detector {i}: Pointing location {place}, Excluded: {excluded !=
85                     0}")
86                 print(grid[place])
87
88             if excluded == 0:
89                 pointing_state[POINTING_PLACE_IDX, i] = place
90                 break
91
92             if place == start_place:
93                 print(f"Warning: Satellite {i} has all pointing vectors excluded.")
94                 pointing_state[POINTING_PLACE_IDX, i] = place
95                 break
96
97 def detectorPointingInitialize(sim_data: Any, grid_points: int) -> None:
98     """
99     Assumes that sim_data.detector is loaded, but the
100     pointing part of detectors is currently empty.
101     Initializes pointing and pointing_state inside detector, and
102     adds a pointing sphere to sim_data.
103     """
104     sensorCount = len(sim_data.detector.filt)
105     generate_pointing_sphere(sim_data, grid_points)
106     detect = sim_data.detector

```

```

107 detect.pointing_state = np.zeros((2, sensorCount), dtype=int)
108 detect.pointing_state[POINTING_COUNT_IDX, :] = grid_points
109 detect.pointing_state[POINTING_PLACE_IDX, :] = np.random.randint(0, grid_points - 1,
110 size=sensorCount)
111 update_detector_pointing(sim_data)
112
113 def demo_exclusion_pointing():
114     """
115     Demonstrates satellite pointing with a solar exclusion angle and a detector
116     field of view, plotting the pointing history on a sphere.
117     """
118     start_time = datetime(2025, 1, 1, 12, 0, 0, tzinfo=timezone.utc)
119     sim_data = create_empty_simulation(start_time)
120     add_celestial_bodies(sim_data)
121
122     # Create a temporary TLE file with a single satellite for this demo
123     single_tle_content = """GEO-SAT
124 1 99999U 99999A 25001.00000000 .00000000 00000-0 00000-0 0 9999
125 2 99999 0.0001 254.9961 0000001 98.4322 261.6813 1.00271900 10""
126
127     temp_tle_path = "temp_single_sat.tle"
128     with open(temp_tle_path, "w") as f:
129         f.write(single_tle_content)
130
131     add_satellites_from_tle(sim_data, temp_tle_path, 'satellites')
132
133     # Clean up the temporary TLE file
134     os.remove(temp_tle_path)
135
136     # Set solar exclusion angle and detector FOV for the single satellite
137     sim_data.detector.solarEx[0] = math.pi/2
138     sim_data.detector.fov[0] = math.pi/5
139
140     # Generate 400 pointing points using the module's generate_pointing_sphere
141     n_points_sphere = 400
142     generate_pointing_sphere(sim_data, n_points_sphere)
143
144     # Initialize pointing_state for the single detector
145     # Assuming the first detector (index 0)
146     sim_data.detector.pointing_state[POINTING_COUNT_IDX, 0] = n_points_sphere
147     sim_data.detector.pointing_state[POINTING_PLACE_IDX, 0] = 0 # Start at the first
148     point
149
150     pointed_directions_history = []
151
152     # Propagate the satellite once to get an initial position
153     sim_data = celestial_update(sim_data, start_time)
154     sim_data = propagate_satellites(sim_data, start_time, 'satellites')
155
156     initial_sat_pos = sim_data.satellites.position[0]
157
158     # --- Plot initialization with unit sphere ---
159     fig = go.Figure()
160     # Add a sphere at the origin to represent the pointing sphere itself for context
161     sphere_radius = 1.0 # Unit sphere
162     u_sphere = np.linspace(0, 2 * np.pi, 50)
163     v_sphere = np.linspace(0, np.pi, 50)
164     x_sphere = sphere_radius * np.outer(np.cos(u_sphere), np.sin(v_sphere))
165     y_sphere = sphere_radius * np.outer(np.sin(u_sphere), np.sin(v_sphere))
166     z_sphere = sphere_radius * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
167     fig.add_trace(go.Surface(
168         x=x_sphere, y=y_sphere, z=z_sphere,
169         colorscale='Blues', showscale=False, opacity=0.1, name='Unit Sphere'
170     ))
171
172     # Add initial satellite position as a marker
173     # The plot_3d_scatter function would normally plot initial_sat_pos scaled,
174     # but here we are just adding it as a marker on the unit sphere for context
175     # as the vectors are also on the unit sphere.
176     initial_sat_direction = initial_sat_pos / np.linalg.norm(initial_sat_pos)
177     fig.add_trace(go.Scatter3d(

```



```

177     x=[initial_sat_direction[0]], y=[initial_sat_direction[1]], z=[
178     initial_sat_direction[2]],
179     mode='markers',
180     marker=dict(size=5, color='blue'),
181     name='Initial Satellite Pointing'
182 ))
183
184 for i in range(400):
185     update_detector_pointing(sim_data, debug=False) # Turn off debug
186     current_pointed_direction = sim_data.detector.pointing[0]
187     snapshot = current_pointed_direction.copy()
188     # print(f"Current pointed direction: {snapshot}") # Print current direction
189     pointed_directions_history.append(snapshot)
190
191     pointed_directions_history = np.array(pointed_directions_history)
192     # print("Array pointedDirectionsHistory", pointed_directions_history)
193
194     # Add the pointing history as markers
195     colors = np.arange(len(pointed_directions_history))
196     fig.add_trace(go.Scatter3d(
197         x=pointed_directions_history[:, 0],
198         y=pointed_directions_history[:, 1],
199         z=pointed_directions_history[:, 2],
200         mode='markers',
201         marker=dict(
202             size=3,
203             color=colors,
204             colorscale='Plasma', # Using a gradient color
205             opacity=0.7
206         ),
207         name='Pointing History'
208     ))
209
210     # Connect the pointing points with lines
211     fig.add_trace(go.Scatter3d(
212         x=pointed_directions_history[:, 0],
213         y=pointed_directions_history[:, 1],
214         z=pointed_directions_history[:, 2],
215         mode='lines',
216         line=dict(color='grey', width=1),
217         name='Pointing Path'
218     ))
219
220     fig.update_layout(
221         title="Detector Pointing with Exclusion",
222         scene=dict(
223             xaxis_title='X (Unit Vector)', yaxis_title='Y (Unit Vector)', zaxis_title='Z
224             (Unit Vector)',
225             aspectmode='data'
226         ),
227         margin=dict(r=20, b=10, l=10, t=40)
228     )
229
230     # print(pointed_directions_history[:100])
231     fig.add_annotation(
232         text="Generated by: demo_exclusion_pointing",
233         xref="paper", yref="paper",
234         x=0.5, y=1.01,
235         showarrow=False,
236         font=dict(size=10, color="gray"),
237         xanchor="center", yanchor="bottom"
238     )
239     return fig
240
241 def jerk(sim_data: Any, satellite_indices: np.ndarray) -> Any:
242     """
243     Moves the pointing vector of specific satellites by 0.3 radians in a
244     random direction.
245
246     This function applies a random rotation to the satellites' pointing vectors.
247
248     Args:
249         sim_data: The main simulation data structure.

```

```

248         satellite_indices: The indices of the satellites to modify.
249
250     Returns:
251         The modified sim_data with the updated pointing vectors.
252     """
253     if satellite_indices.size == 0:
254         return sim_data
255
256     p = sim_data.detector.pointing[satellite_indices]
257     p_norm = p / np.linalg.norm(p, axis=1)[:, np.newaxis]
258
259     # Generate a random vector not parallel to p_norm
260     r = np.random.randn(*p.shape)
261     r -= np.sum(r * p_norm, axis=1)[:, np.newaxis] * p_norm
262     k_hat = r / np.linalg.norm(r, axis=1)[:, np.newaxis]
263
264     theta = 0.3
265     cos_theta = np.cos(theta)
266     sin_theta = np.sin(theta)
267
268     # Rodrigues' rotation formula
269     p_new = p_norm * cos_theta + np.cross(k_hat, p_norm) * sin_theta
270
271     sim_data.detector.pointing[satellite_indices] = p_new
272
273     return sim_data

```

Listing 10.1: pointing.py Source Code

Chapter 11

scandetectors.py: Target Detection & Photometric Engine

Overview

Vectorized detection engine calculating signal, background noise, and SNR for visible targets.

Literate Source Code

The source code of `scandetectors.py` is documented below. Section label: 11.

```
1
2 import numpy as np
3 from lambertian import lambertiansphere, includedAngle
4 import radiometry_calcs
5 from constants import EARTH_RADIUS
6
7
8 def get_spherical_coords(arr: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
9     """
10     Converts 3D Cartesian coordinates to spherical angles (theta, phi).
11
12     Args:
13         arr: An (N, 3) NumPy array of Cartesian coordinates.
14
15     Returns:
16         tuple[np.ndarray, np.ndarray]:
17             - theta: The inclination angle from the z-axis in radians [0, pi].
18             - phi: The azimuthal angle in the x-y plane in radians [-pi, pi].
19     """
20     x, y, z = arr[:, 0], arr[:, 1], arr[:, 2]
21     r = np.linalg.norm(arr, axis=1)
22     # theta: angle from the z-axis [0, pi]
23     # We use np.divide to handle potential division by zero if r is 0
24     theta = np.arccos(np.clip(z / r, -1.0, 1.0))
25     # phi: angle in the x-y plane [-pi, pi]
26     phi = np.arctan2(y, x)
27     return theta, phi
28
29 #####
30
31
32 def scandetectors(sim_data: dict, print_output: int = 0, mask: np.ndarray = None):
33     """
34     Scans for and processes detector data within the simulation.
35
36     This function orchestrates the process of determining which fixed targets
37     are visible to each satellite's detector and calculates the resulting
38     signal-to-noise ratio (SNR).
39
40     The general flow involves:
41     1. Data Extraction: Relevant simulation parameters such as satellite positions,
```

```

42     detector characteristics (pointing, FOV, integration time, aperture area),
43     target properties (positions, sizes, albedo), and celestial body positions
44     (e.g., Sun) are extracted from the 'sim_data' dictionary. If a 'mask' is
45     provided, only the specified subset of detectors is extracted.
46 2. **Background Flux Calculation**: Utilizes 'radiometry_calcs.fluxes' to determine
47     the background contributions from the Sun, space, and sky based on the
48     detector's filter.
49 3. **Visibility Determination**: For the active satellites, the function calculates
50     the angular separation between each detector's pointing vector and the
51     vectors to all fixed targets using vectorized broadcasting. A 2D mask is then
52     applied to identify only those target-detector pairs that fall within the
53     respective fields of view (FOV).
54 4. **Detector Flux Calculation**: For the visible targets, the incident flux
55     on the detector is calculated using the 'lambertian.lambertiansphere' model
56     in a vectorized manner. This calculation considers the Sun's position, the
57     target's position, albedo, radius, and the solar flux.
58 5. **SNR Calculation**: Based on the calculated detector flux, the integration time
59     ,
60     and the detector's aperture area, the signal and noise levels are determined
61     simultaneously for all detections. The signal-to-noise ratio (SNR) is then
62     computed from these values.
63 6. **Output/Storage**: The calculated signal, noise, and SNR for each detected
64     target are returned in a dictionary. If 'print_output' is enabled, they are
65     also printed to the console.
66
67 Args:
68     sim_data (dict): The main simulation data dictionary.
69     print_output (int): If > 0, prints detailed results for each detection.
70     mask (np.ndarray, optional): A boolean mask of length equal to the total number
71         of satellites, indicating which detectors to
72         process.
73         If None, all detectors are processed.
74
75 Returns:
76     dict: A dictionary containing simulation 'time' and arrays for 'sat_indices',
77         'target_indices', 'signal', 'noise', and 'snr'.
78
79 TODO: we are using the same filter for all the detectors!
80 """
81
82 # 1. Data Extraction and Masking
83 num_sats = sim_data.counts.get('satellites', 0)
84 num_obs = sim_data.counts.get('observatories', 0)
85 num_total_detectors = len(sim_data.detector.filt)
86 sim_time = sim_data.time
87
88 if mask is None:
89     mask = np.ones(num_total_detectors, dtype=bool)
90
91 active_indices = np.where(mask)[0]
92
93 # Initialize empty result structure
94 results = {
95     'time': sim_time,
96     'sat_indices': np.array([], dtype=int),
97     'target_indices': np.array([], dtype=int),
98     'signal': np.array([], dtype=float),
99     'noise': np.array([], dtype=float),
100     'snr': np.array([], dtype=float)
101 }
102
103 if len(active_indices) == 0:
104     return results
105
106 # Subset observer positions and detector properties cleanly
107 satpositions = sim_data.get_detector_positions(mask)
108 det_subset = sim_data.detector.subset(mask)
109
110 detectorVect = det_subset.pointing
111 fovs = det_subset.fov
112 integrationTime = det_subset.integrationTime
113 apertureArea = det_subset.apertureArea
114 pixelOmega = det_subset.pixelOmega
115 qe = det_subset.qe

```

```

113
114 # Target and celestial data
115 targets = sim_data.fixedpoints.position
116 sunVect = sim_data.celestial.position[0]
117 albedo = sim_data.fixedpoints.albedo
118 radius = sim_data.fixedpoints.size / 2
119
120 # 2. Background Flux Calculation
121 # Assuming same filter for active group (as per existing TODO)
122 filter_name = sim_data.detector.filt[active_indices[0]]
123 sun, space, sky = radiometry_calcs.fluxes(filter_name)
124
125 if print_output:
126     print(f'Using filter: {filter_name}')
127     print(f'sun, space, sky \n      {sun:.3e}, {space:.3e}, {sky:.3e}')
128
129 # 3. Visibility Determination (Vectorized Geometry)
130 # toTargets shape: (num_active_sats, num_targets, 3)
131 toTargets = targets[np.newaxis, :, :] - satpositions[:, np.newaxis, :]
132
133 # Calculate norms for normalization and distance
134 norms_toTargets = np.linalg.norm(toTargets, axis=2)
135 norms_toTargets_safe = np.where(norms_toTargets == 0, 1.0, norms_toTargets)
136
137 # Normalized vectors from sats to targets
138 normalized_toTargets = toTargets / norms_toTargets_safe[:, :, np.newaxis]
139
140 # Vectorized dot product for angles: (num_active_sats, num_targets)
141 dot_products = np.einsum('si,sti->st', detectorVect, normalized_toTargets)
142     # This line uses NumPy's Einstein Summation (einsum) to perform a
143     # high-performance, vectorized dot product between the satellite
144     # pointing vectors and the vectors to every target.
145 angles = np.arccos(np.clip(dot_products, -1.0, 1.0))
146
147 # Apply FOV mask: (num_active_sats, num_targets)
148 # fov is the full field-of-view diameter; a target is visible
149 # when its angular offset from boresight is less than fov / 2.
150 visible_mask = angles < (fovs[:, np.newaxis] / 2)
151 sat_hit_idx, target_hit_idx = np.where(visible_mask)
152
153 if len(sat_hit_idx) == 0:
154     if print_output:
155         print("No targets visible to active detectors.")
156     return results
157
158 # 3.5 Occlusion Filter (Earth Limb & Horizon)
159 hit_obs_pos = satpositions[sat_hit_idx]
160 hit_toTargets_pre = toTargets[sat_hit_idx, target_hit_idx]
161 hit_norms_pre = norms_toTargets[sat_hit_idx, target_hit_idx]
162
163 u_hit_toTargets = hit_toTargets_pre / hit_norms_pre[:, np.newaxis]
164 obs_norms = np.linalg.norm(hit_obs_pos, axis=1)
165 valid_obs = obs_norms > 1e-9
166
167 u_nadir = np.zeros_like(hit_obs_pos)
168 u_nadir[valid_obs] = -hit_obs_pos[valid_obs] / obs_norms[valid_obs, np.newaxis]
169
170 cos_angle_to_nadir = np.sum(u_hit_toTargets * u_nadir, axis=1)
171 angle_to_nadir = np.arccos(np.clip(cos_angle_to_nadir, -1.0, 1.0))
172
173 hit_earthEx = det_subset.earthEx[sat_hit_idx]
174 hit_categories = np.array(det_subset.category)[sat_hit_idx]
175
176 obs_is_ground = (hit_categories == 'observatories')
177
178 # Ground observatory horizon check: angle from zenith must be <= (90deg -
179 min_elevation)
180 angle_to_zenith = np.pi - angle_to_nadir
181 max_zenith = np.pi / 2.0 - hit_earthEx
182 horizon_occluded = obs_is_ground & valid_obs & (angle_to_zenith > max_zenith)
183
184 # Spacecraft Earth limb check: angle to Earth center must be >= (apparent_radius +
185 earthEx)

```

```

184     safe_obs_norms = np.where(obs_norms > EARTH_RADIUS, obs_norms, EARTH_RADIUS)
185     apparent_radius = np.where(
186         obs_norms > EARTH_RADIUS,
187         np.arcsin(EARTH_RADIUS / safe_obs_norms),
188         np.pi / 2.0
189     )
190     limb_occluded = (~obs_is_ground) & valid_obs & (angle_to_nadir < (apparent_radius +
191         hit_earthEx))
192
193     is_occluded = horizon_occluded | limb_occluded
194
195     # Filter out occluded hits
196     clear_hits = ~is_occluded
197     sat_hit_idx = sat_hit_idx[clear_hits]
198     target_hit_idx = target_hit_idx[clear_hits]
199
200     if len(sat_hit_idx) == 0:
201         if print_output:
202             print("No targets visible (all occluded) to active detectors.")
203         return results
204
205     # 4. Detector Flux Calculation (Vectorized Radiometry)
206     # Subset data for specific detector-target pairs (hits)
207     hit_toTargets = toTargets[sat_hit_idx, target_hit_idx]
208     hit_norms = norms_toTargets[sat_hit_idx, target_hit_idx]
209     hit_albedo = albedo[target_hit_idx]
210     hit_radius = radius[target_hit_idx]
211
212     # Lambertian Phase Angle Calculation
213     vec_from_sphere_to_observer = -hit_toTargets
214     vec_from_sphere_to_light = np.tile(sunVect, (len(sat_hit_idx), 1))
215
216     angle_light_observer = includedAngle(vec_from_sphere_to_light,
217         vec_from_sphere_to_observer)
218
219     target_brightness = lambertiansphere(
220         angle_light_observer,
221         hit_albedo,
222         hit_radius,
223         sun,
224         debug=0
225     )
226
227     detectorFlux = target_brightness / (np.pi * hit_norms ** 2)
228
229     # 5. SNR Calculation
230     hit_itime = integrationTime[sat_hit_idx]
231     hit_aper = apertureArea[sat_hit_idx]
232     hit_qe = qe[sat_hit_idx]
233     hit_omega = pixelOmega[sat_hit_idx]
234     hit_photoEff = sim_data.detector.photoEff[mask][sat_hit_idx]
235
236     signal = detectorFlux * hit_itime * hit_aper * hit_qe * hit_photoEff
237     noise = np.sqrt(space * hit_itime * hit_aper * hit_omega * hit_qe)
238     snr = signal / noise
239
240     # 6. Output/Storage
241     if print_output:
242         for k in range(len(sat_hit_idx)):
243             i = active_indices[sat_hit_idx[k]]
244             j = target_hit_idx[k]
245             print(f"Sat {i} (itime={hit_itime[k]:.1f}s) detects Target {j}:")
246             print(f"  Signal: {signal[k]:12.3e} | Noise: {noise[k]:12.3e} | SNR: {snr[k]:12.3e}")
247
248     results.update({
249         'sat_indices': active_indices[sat_hit_idx],
250         'target_indices': target_hit_idx,
251         'signal': signal,
252         'noise': noise,
253         'snr': snr
254     })

```

254

```
return results
```

Listing 11.1: scandetectors.py Source Code

Chapter 12

radiometry_data.py: Filter Spectral Data

Overview

Defines FILTER_DATA dictionary containing astronomical filter zero-points, sky/space background, and solar magnitudes.

Literate Source Code

The source code of radiometry_data.py is documented below. Section label: 12.

```
1 import scipy.constants as const
2
3 # --- Physical Constants ---
4 AU_M = const.au # Astronomical Unit in meters
5 RSUN_M = 6.957e8 # Radius of the Sun in meters
6
7 # --- Radiometric Data ---
8
9 # Data is compiled for standard astronomical filters.
10 #
11 # Magnitudes are apparent magnitudes in the AB or Vega system.
12 # sun is the apparent magnitude of the sun in the given filter band.
13 # Sky brightness is for a dark site at new moon, in magnitudes per square arcsecond.
14 #     For space-based IR telescopes, this is the zodiacal + telescope background.
15 #     For ground-based IR, it's dominated by thermal emission.
16 # Wavelengths and bandwidths are in nanometers (nm).
17 # Zero point is the photon flux for a 0-magnitude object in photons per second per
    square meter.
18
19
20
21 FILTER_DATA = {
22     # Johnson-Cousins UBVRI Filters
23
24     'U': {
25         'sun': -26.03,
26         'sky': 22.0,
27         'space': 22.9,
28         'central_wavelength': 365.0,
29         'bandwidth': 66.0,
30         'zero_point': 4.96e9,
31     },
32     'B': {
33         'sun': -26.13,
34         'sky': 22.7,
35         'space': 23.3,
36         'central_wavelength': 445.0,
37         'bandwidth': 94.0,
38         'zero_point': 1.36e10,
39     },
```



```

40     'V': {
41         'sun': -26.78,
42         'sky': 21.9, # Mathematica was assuming 21.61, pretty close actually!
43         'space': 23,
44         'central_wavelength': 551.0,
45         'bandwidth': 88.0,
46         'zero_point': 8.79e9,
47     },
48     'R': {
49         'sun': -27.11,
50         'sky': 21.0,
51         'space': 22.0,
52         'central_wavelength': 658.0,
53         'bandwidth': 138.0,
54         'zero_point': 9.74e9,
55     },
56     'I': {
57         'sun': -27.47,
58         'sky': 20.0,
59         'space': 21.2,
60         'central_wavelength': 806.0,
61         'bandwidth': 149.0,
62         'zero_point': 7.11e9,
63     },
64     # Near-Infrared JHK Filters
65     'J': {
66         'sun': -27.91,
67         'sky': 16.0,
68         'space': 21.5,
69         'central_wavelength': 1235.0,
70         'bandwidth': 162.0,
71         'zero_point': 3.15e9,
72     },
73     'H': {
74         'sun': -28.25,
75         'sky': 14.0,
76         'space': 20.5,
77         'central_wavelength': 1646.0,
78         'bandwidth': 251.0,
79         'zero_point': 2.35e9,
80     },
81     'K': {
82         'sun': -28.29,
83         'sky': 13.0,
84         'space': 19.5,
85         'central_wavelength': 2159.0,
86         'bandwidth': 262.0,
87         'zero_point': 1.17e9,
88     },
89     # SDSS Filters
90     'g': {
91         'sun': -26.47,
92         'sky': 21.8,
93         'space': 23.0,
94         'central_wavelength': 477.0,
95         'bandwidth': 137.9,
96         'zero_point': 1.582e10,
97     },
98     'r': {
99         'sun': -26.93,
100        'sky': 20.8,
101        'space': 22.2,
102        'central_wavelength': 623.1,
103        'bandwidth': 138.2,
104        'zero_point': 1.214e10,
105    },
106    'i': {
107        'sun': -27.05,
108        'sky': 20.2,
109        'space': 21.5,
110        'central_wavelength': 762.5,
111        'bandwidth': 153.5,
112        'zero_point': 1.103e10,

```

```

113 },
114 'z': {
115     'sun': -27.07,
116     'sky': 19.0,
117     'space': 20.8,
118     'central_wavelength': 913.4,
119     'bandwidth': 140.9,
120     'zero_point': 8.455e9,
121 },
122 # Ground-Based Mid-IR Filters
123 'L': {
124     'sun': None,
125     'sky': 3.5,
126     'central_wavelength': 3500.0,
127     'bandwidth': 600.0,
128     'zero_point': 9.4e9,
129 },
130 'M': {
131     'sun': None,
132     'sky': 0.0,
133     'central_wavelength': 4800.0,
134     'bandwidth': 600.0,
135     'zero_point': 6.8e9,
136 },
137 'N': {
138     'sun': None,
139     'sky': -6.0,
140     'central_wavelength': 10200.0,
141     'bandwidth': 5000.0,
142     'zero_point': 2.7e10,
143 },
144 # JWST MIRI Filters
145 'F560W': { 'sun': None,
146             'sky': 24.4,
147             'central_wavelength': 5600.0,
148             'bandwidth': 1000.0,
149             'zero_point': 9.78e9 },
150 'F770W': { 'sun': None,
151             'sky': 24.4,
152             'central_wavelength': 7700.0,
153             'bandwidth': 1950.0,
154             'zero_point': 1.39e10 },
155 'F1000W': { 'sun': None,
156              'sky': 23.6,
157              'central_wavelength': 10000.0,
158              'bandwidth': 1800.0,
159              'zero_point': 9.86e9 },
160 'F1500W': { 'sun': None,
161              'sky': 21.9,
162              'central_wavelength': 15000.0,
163              'bandwidth': 2940.0,
164              'zero_point': 1.08e10 },
165 'F2550W': { 'sun': None,
166              'sky': 19.4,
167              'central_wavelength': 25500.0,
168              'bandwidth': 4050.0,
169              'zero_point': 8.70e9 }
170 }

```

Listing 12.1: radiometry_data.py Source Code

Chapter 13

radiometry_calcs.py: Radiometric Calculations & Blackbody

Overview

Computes photon fluxes, magnitude conversions, and Planck blackbody spectral radiance.

Literate Source Code

The source code of `radiometry_calcs.py` is documented below. Section label: [13](#).

```
1 import math
2 import scipy.integrate
3 import scipy.constants as const
4 import numpy as np
5 import plotly.graph_objects as go
6 from radiometry_data import *
7 from constants import ARCSEC
8
9 def fluxes(band):
10     """
11     uses the FILTER_DATA table from radiometry_data.py for data
12     Looks up information based on the argument band, which is
13     usually something like an astronomical band... U, B, V, etc.
14     It returns three numbers:
15     sun which is the solar flux at earth in photons/s/m^2
16     sky which is the sky brightness at earth in p/s/steradian/m^2
17     space, sky brightness in space in p/s/steradian/m^2
18     """
19     x = FILTER_DATA[band]
20     zp = x['zero_point']
21     sun_mag = x.get('sun')
22     sun = amag(sun_mag) * zp if sun_mag is not None else 0.0
23
24     space_mag = x.get('space', x.get('sky'))
25     space = amag(space_mag) * zp / (ARCSEC**2) if space_mag is not None else 0.0
26
27     sky_mag = x.get('sky')
28     sky = amag(sky_mag) * zp / (ARCSEC**2) if sky_mag is not None else 0.0
29
30     return (sun, space, sky)
31
32
33 def mag(x: float) -> float:
34     """
35     Calculates a magnitude value from a linear ratio.
36     Uses the formula: magnitude = -2.5 * log10(ratio)
37     """
38     if x <= 0:
39         raise ValueError("Input for mag() must be positive.")
40     return -2.5 * math.log10(x)
41
```

```

42 def amag(x: float) -> float:
43     """
44     Calculates the linear ratio from a magnitude value.
45     This is the inverse of the mag() function.
46     Uses the formula: ratio = 10**(-0.4 * magnitude)
47     """
48     return 10**(-0.4 * x)
49
50
51 def _planck_law(wav_m: float, temp_k: float) -> float:
52     """
53     Helper function for Planck's law for spectral radiance.
54     Args:
55         wav_m: Wavelength in meters.
56         temp_k: Temperature in Kelvin.
57     Returns:
58         Spectral radiance in W / (m^2 * sr * m).
59     """
60     if wav_m <= 0:
61         return 0.0
62     exponent = (const.h * const.c) / (wav_m * const.k * temp_k)
63     if exponent > 700:
64         return 0.0
65     numerator = 2.0 * const.h * const.c**2
66     denominator = (wav_m**5) * (math.exp1(exponent))
67     if denominator == 0:
68         return 0.0
69     return numerator / denominator
70
71 def blackbody_flux(
72     temperature: float,
73     lambda_short: float,
74     lambda_long: float
75 ) -> float:
76     """
77     Numerically computes the integrated spectral radiance of
78     a blackbody over a given wavelength band.
79
80     Requires the scipy library.
81
82     Args:
83         temperature: The temperature of the blackbody in Kelvin.
84         lambda_short: The short wavelength of the band in microns.
85         lambda_long: The long wavelength of the band in microns.
86
87     Returns:
88         The integrated spectral radiance in units of
89         Watts / (m^2 * steradian).
90     """
91     if lambda_short >= lambda_long:
92         raise ValueError("lambda_short must be less than lambda_long.")
93     lambda_short_m = lambda_short * 1e-6
94     lambda_long_m = lambda_long * 1e-6
95     integrated_radiance, _ = scipy.integrate.quad(
96         lambda wav: _planck_law(wav, temperature),
97         lambda_short_m,
98         lambda_long_m
99     )
100     return integrated_radiance
101
102 def stefan_boltzmann_law(temperature: float) -> float:
103     """
104     Calculates the total power radiated per unit area by a
105     blackbody using the Stefan-Boltzmann law.
106
107     Args:
108         temperature: The temperature of the blackbody in Kelvin.
109
110     Returns:
111         The total radiated power per unit area in W / m^2.
112     """
113     return const.sigma * (temperature**4)
114

```

```

115 def plot_blackbody_spectrum(temperature: float):
116     """
117     Plots the spectral radiance of a blackbody from
118     0.5 to 30 microns.
119
120     Args:
121         temperature: The temperature of the blackbody in Kelvin.
122     """
123     wavelengths_microns = np.geomspace(0.5, 30, 500)
124     wavelengths_m = wavelengths_microns * 1e-6
125     spectral_radiance = [_planck_law(w, temperature) * 1e-6 for w in wavelengths_m]
126     fig = go.Figure()
127     fig.add_trace(go.Scatter(
128         x=wavelengths_microns,
129         y=spectral_radiance,
130         mode='lines',
131         name=f'{temperature} K Blackbody'
132     ))
133     fig.update_layout(
134         title_text=f'Blackbody Spectrum ({temperature} K)',
135         xaxis_title="Wavelength (microns)",
136         yaxis_title="Spectral Radiance (W / m^2 / sr / micron)",
137         xaxis_type="log",
138         yaxis_type="log",
139         template="plotly_white"
140     )
141     fig.show()
142
143 def plot_blackbody_spectrum_visible_nir(temperature: float):
144     """
145     Plots the spectral radiance of a blackbody from
146     0.1 to 1 micron.
147
148     Args:
149         temperature: The temperature of the blackbody in Kelvin.
150     """
151     wavelengths_microns = np.linspace(0.1, 1.0, 500)
152     wavelengths_m = wavelengths_microns * 1e-6
153     spectral_radiance = [_planck_law(w, temperature) * 1e-6 for w in wavelengths_m]
154     fig = go.Figure()
155     fig.add_trace(go.Scatter(
156         x=wavelengths_microns,
157         y=spectral_radiance,
158         mode='lines',
159         name=f'{temperature} K Blackbody'
160     ))
161     fig.update_layout(
162         title_text=f'Blackbody Spectrum ({temperature} K) - Visible/NIR',
163         xaxis_title="Wavelength (microns)",
164         yaxis_title="Spectral Radiance (W / m^2 / sr / micron)",
165         template="plotly_white"
166     )
167     fig.show()

```

Listing 13.1: radiometry_calcs.py Source Code

Chapter 14

lambertian.py: Lambertian Sphere Brightness

Overview

Calculates phase-angle dependent reflected flux from Lambertian target spheres.

Literate Source Code

The source code of `lambertian.py` is documented below. Section label: [14](#).

```
1 import numpy as np
2
3 def lambertiansphere(
4     angle_light_observer: np.ndarray,
5     albedo: np.ndarray,
6     radius: np.ndarray,
7     base_brightness,
8     debug: int = 0
9 ) -> np.ndarray:
10     """
11     Calculates the emitted brightness from multiple lambertian spheres
12     based on phase angle. This function computes the brightness emitted
13     *from the sphere's surface* in the direction of the observer.
14     The final apparent brightness at the observer's location must be
15     calculated by dividing this result by  $\pi * (\text{distance\_to\_observer})^2$ 
16     to satisfy conservation of energy for a diffuse sphere.
17
18     Args:
19         angle_light_observer: A 1D NumPy array of shape (N,) with the
20             phase angle in radians for each sphere. This is the angle
21             between the light source and the observer as seen from
22             the sphere's center.
23         albedo: A 1D NumPy array of shape (N,) with the fraction of
24             incident light that is reflected for each sphere (0.0 to 1.0).
25         radius: A 1D NumPy array of shape (N,) with the radius of each
26             sphere in meters.
27         base_brightness: A 1D NumPy array of shape (N,) with the incident
28             flux or brightness of the light source at each sphere's
29             location.
30         debug: An optional integer. If set to 1, a table of inputs and
31             outputs will be printed before the function returns.
32             Defaults to 0.
33
34     Returns:
35         A 1D NumPy array of shape (N,) containing the brightness emitted
36         from each sphere's surface (e.g., in Watts per steradian per
37         square meter). To get apparent brightness at the observer,
38         divide by  $\pi * (\text{distance\_to\_observer})^2$ .
39     """
40     if not np.all((albedo >= 0.0) & (albedo <= 1.0)):
41         raise ValueError("All albedo values must be between 0.0 and 1.0.")
```

```

42 if np.any(radius < 0):
43     raise ValueError("Radius cannot be negative.")
44 if (not isinstance(angle_light_observer, np.ndarray) or
45     angle_light_observer.ndim != 1):
46     raise ValueError("angle_light_observer must be a 1D NumPy array.")
47
48 # The phase angle is physically constrained to be between 0 and pi.
49 # We clip the value to handle out-of-range inputs gracefully.
50 alpha = np.clip(angle_light_observer, 0, np.pi)
51
52 term1 = np.sin(alpha)
53 term2 = (np.pi - alpha) * np.cos(alpha)
54 phase_function_value = (2 / (3 * np.pi)) * (term1 + term2)
55
56 cross_sectional_area = np.pi * (radius ** 2)
57
58 effective_cross_section = (
59     albedo *
60     cross_sectional_area *
61     phase_function_value
62 )
63
64 # The apparent brightness is the incident brightness multiplied
65 # by the effective reflecting area.
66 # The division by distance squared will be handled by the caller,
67 # as instructed by the user, to accommodate the 4*pi*r^2 factor.
68 apparent_brightness = (base_brightness * effective_cross_section)
69
70 if debug == 1:
71     print("\n--- Debug Info: lambertiansphere ---")
72     print("Effective Cross Section: ", effective_cross_section)
73     num_spheres = len(albedo)
74     h1 = f"{'Index':<5} {'Phase Angle (rad)':<18} {'Albedo':<10}"
75     h2 = f"{'Radius (m)':<12} {'Base Brightness':<18} {'Emitted Brightness':<22}"
76     header = h1 + " " + h2
77     print(header)
78     print("-" * len(header))
79
80     display_limit = 5
81     for i in range(num_spheres):
82         if (num_spheres > 2 * display_limit and
83             display_limit <= i < num_spheres - display_limit):
84             if i == display_limit:
85                 p1 = f"{'':<5} {'...':<18} {'...':<10}"
86                 p2 = f"{'...':<12} {'...':<18} {'...':<22}"
87                 print(p1 + " " + p2)
88             continue
89
90         brightness_val = (
91             base_brightness[i]
92             if np.ndim(base_brightness) > 0
93             else base_brightness
94         )
95         r1 = f"{i:<5} {angle_light_observer[i]:<18.4e} {albedo[i]:<10.4f}"
96         r2 = f"{radius[i]:<12.4e} {brightness_val:<18.4e}"
97         r3 = f"{apparent_brightness[i]:<22.4e}"
98         print(f"{r1} {r2} {r3}")
99     print("-----\n")
100 elif debug == 2:
101     print("\n--- Detailed Debug Info: lambertiansphere ---")
102     num_spheres = len(albedo)
103     header = (
104         f"{'Index':<5} {'Input Angle':<15} {'Clipped Alpha':<15} "
105         f"{'Phase Func Val':<15} {'Cross Sect Area':<15} "
106         f"{'Effective CS':<15} {'Albedo':<10} {'Radius (m)':<12} "
107         f"{'Base Brightness':<18} {'Emitted Brightness':<22}"
108     )
109     print(header)
110     print("-" * len(header))
111
112     display_limit = 5
113     for i in range(num_spheres):
114         if (num_spheres > 2 * display_limit and

```

```

115         display_limit <= i < num_spheres - display_limit):
116             if i == display_limit:
117                 dots = ["..."] * 9
118                 print(f"{'.':<5} " + " ".join([f"{d:<15}" for d in dots]))
119                 continue
120
121             brightness_val = (
122                 base_brightness[i]
123                 if np.ndim(base_brightness) > 0
124                 else base_brightness
125             )
126             print(
127                 f"{i:<5} {angle_light_observer[i]:<15.4e} {alpha[i]:<15.4e} "
128                 f"{phase_function_value[i]:<15.4e} "
129                 f"{cross_sectional_area[i]:<15.4e} "
130                 f"{effective_cross_section[i]:<15.4e} {albedo[i]:<10.4f} "
131                 f"{radius[i]:<12.4e} {brightness_val:<18.4e} "
132                 f"{apparent_brightness[i]:<22.4e}"
133             )
134             print("-----\n")
135
136         return apparent_brightness
137
138
139 def simple_lambertian(
140     diameter: float,
141     distance: float,
142     albedo: float,
143     angle: float,
144     base_brightness: float
145 ) -> float:
146     """
147     Calculates the apparent brightness of a Lambertian sphere.
148     See Also lambertiansphere, which works with complete vectors
149     of objects in a more code friendly way!
150
151     This function computes the apparent brightness of a diffusely
152     reflecting sphere based on its physical properties, viewing
153     geometry, and a given base incident brightness. It simplifies
154     the calculation by taking the phase angle directly, rather than
155     calculating it from vectors.
156
157     Args:
158         diameter: The diameter of the sphere in meters.
159         distance: The distance from the sphere to the observer
160                 in meters.
161         albedo: The fraction of incident light that is
162                reflected (0.0 to 1.0).
163         angle: The phase angle in radians. This is the angle
164               between the light source and the observer as seen
165               from the sphere's center (expected to be between 0 and pi).
166         base_brightness: The incident flux or brightness of the
167                        light source at the sphere's location (e.g., in
168                        Watts per square meter or photons / s / m^2).
169
170     Returns:
171         The apparent brightness of the sphere as observed from
172         the specified distance (e.g., in Watts per square meter
173         or photons / s / m^2).
174     """
175     if not 0.0 <= albedo <= 1.0:
176         raise ValueError("Albedo must be between 0.0 and 1.0.")
177     if diameter < 0:
178         raise ValueError("Diameter cannot be negative.")
179     if distance <= 0:
180         raise ValueError("Distance must be positive.")
181
182     # Calculate radius from diameter
183     radius = diameter / 2.0
184
185     # The phase angle is physically constrained to be between 0 and pi.
186     # We clip the value to handle out-of-range inputs gracefully.
187     angle = np.clip(angle, 0, np.pi)

```



```

188
189 # This is the phase function for a Lambertian sphere.
190 # It describes how the brightness changes with the phase angle.
191 term1 = np.sin(angle)
192 term2 = (np.pi - angle) * np.cos(angle)
193 phase_function_value = (2 / (3 * np.pi)) * (term1 + term2)
194
195 # Cross-sectional area of the sphere.
196 cross_sectional_area = np.pi * (radius ** 2)
197
198 # The effective cross-section combines the sphere's size,
199 # reflectivity (albedo), and the phase function.
200 effective_cross_section = (
201     albedo *
202     cross_sectional_area *
203     phase_function_value
204 )
205
206 # The apparent brightness is the incident brightness multiplied
207 # by the effective reflecting area, with the resulting light
208 # distributed according to the inverse square law.
209 apparent_brightness = (
210     (base_brightness * effective_cross_section) /
211     (np.pi * distance ** 2)
212 )
213
214 return apparent_brightness
215
216
217 def includedAngle(
218     vectors1: np.ndarray,
219     vectors2: np.ndarray
220 ) -> np.ndarray:
221     """
222     Calculates the included angle in radians between corresponding vectors
223     in two input NumPy arrays.
224
225     Args:
226         vectors1 (np.ndarray): A NumPy array of shape (N, 3) representing the
227                                first set of vectors.
228         vectors2 (np.ndarray): A NumPy array of shape (N, 3) representing the
229                                second set of vectors.
230
231     Returns:
232         np.ndarray: A 1D NumPy array of shape (N,) containing the included
233                    angle in radians for each corresponding pair of vectors.
234     """
235     if vectors1.shape != vectors2.shape:
236         raise ValueError("Input arrays must have the same shape.")
237     if vectors1.shape[1] != 3:
238         raise ValueError("Input vectors must be 3-dimensional.")
239
240     from minimalsimulation import safe_normalize
241     unit_vectors1 = safe_normalize(vectors1, axis=1)
242     unit_vectors2 = safe_normalize(vectors2, axis=1)
243
244     # Calculate the dot product of the normalized vectors
245     dot_product = np.einsum('ij,ij->i', unit_vectors1, unit_vectors2)
246
247     # Clip values to ensure they are within the valid range for arccos (-1 to 1)
248     dot_product = np.clip(dot_product, -1.0, 1.0)
249
250     # Calculate the angle in radians
251     angles = np.arccos(dot_product)
252
253     return angles

```

Listing 14.1: lambertian.py Source Code

Chapter 15

dataHandling.py: Simulation Data Logging & Analysis

Overview

Collects observation results into DataHandler, supporting Pandas DataFrame conversion, CSV/Parquet export, and gap-time analysis.

Literate Source Code

The source code of dataHandling.py is documented below. Section label: 15.

```
1 import pandas as pd
2 import numpy as np
3 import plotly.graph_objects as go
4
5 class DataHandler:
6     """
7     A class to collect, manage, and export simulation results from VibeVolts.
8
9     This handler facilitates collecting results from multiple calls to
10    'nextIntegration' (via 'scandetectors') and converting them into a
11    structured Pandas DataFrame for analysis, plotting, and export.
12
13    Functions:
14    - __init__(): Initializes an empty results collection.
15    - add_results(results): Appends a dictionary of scan results to the collection.
16    - get_dataframe(): Consolidates all results into a single Pandas DataFrame.
17    - save_to_csv(filename): Exports the collected data to a CSV file.
18    - save_to_parquet(filename): Exports the collected data to a Parquet file.
19
20    import pandas as pd
21    df = pd.read_parquet('my_data.parquet')
22    - clear(): Resets the handler by removing all collected data.
23    """
24
25    def __init__(self):
26        """Initializes an empty list to store result dictionaries."""
27        self.results_list = []
28        self._master_df = None
29
30    def add_results(self, results: dict):
31        """
32        Appends a result dictionary from nextIntegration to the collection.
33
34        Args:
35            results (dict): The dictionary returned by nextIntegration/scandetectors.
36                           Expected to contain 'time', 'sat_indices',
37                           'target_indices', 'signal', 'noise', and 'snr'.
38
39        """
40        # We only add if there are actual detections to keep the DataFrame concise.
41        # If you need to track empty time steps, remove this check.
42        if len(results['sat_indices']) > 0:
```

```

41         # Converting to DataFrame immediately ensures consistent types
42         df_step = pd.DataFrame(results)
43         self.results_list.append(df_step)
44         # Reset master_df so it's recalculated on next access
45         self._master_df = None
46
47     def get_dataframe(self) -> pd.DataFrame:
48         """
49         Combines all collected results into a single Pandas DataFrame.
50
51         Returns:
52             pd.DataFrame: A DataFrame with columns for time, satellite indices,
53                           target indices, signal, noise, and SNR.
54                           Returns an empty DataFrame if no results have been added.
55         """
56         if not self.results_list:
57             return pd.DataFrame()
58
59         if self._master_df is None:
60             self._master_df = pd.concat(self.results_list, ignore_index=True)
61
62         return self._master_df
63
64     def save_to_csv(self, filename: str):
65         """
66         Saves the collected results to a CSV file.
67
68         Args:
69             filename (str): The path/name of the file to save (e.g., 'results.csv').
70         """
71         df = self.get_dataframe()
72         if not df.empty:
73             df.to_csv(filename, index=False)
74             print(f"Results saved to {filename}")
75         else:
76             print("No data to save.")
77
78     def save_to_parquet(self, filename: str):
79         """
80         Saves the collected results to a Parquet file (more efficient than CSV).
81
82         Args:
83             filename (str): The path/name of the file to save (e.g., 'results.parquet').
84         """
85         df = self.get_dataframe()
86         if not df.empty:
87             df.to_parquet(filename, index=False)
88             print(f"Results saved to {filename}")
89         else:
90             print("No data to save.")
91
92     def clear(self):
93         """Clears all collected results to start a new run."""
94         self.results_list = []
95         self._master_df = None
96
97     def calculate_gap_times(
98         self, target_id: int = None, pooled: bool = False
99     ) -> dict[int, np.ndarray] | np.ndarray:
100         """
101         Calculates the time gaps (in seconds) between consecutive observations of
102         targets.
103
104         If target_id is specified, returns a 1D numpy array of gap times (in seconds)
105         for that target.
106         If target_id is None and pooled is True, returns a single 1D numpy array
107         containing
108         the gap times of all targets concatenated.
109         If target_id is None and pooled is False, returns a dictionary mapping target
110         indices to their respective numpy arrays of gap times.
111
112         Args:
113             target_id (int, optional): The index of a specific target. Defaults to None.

```

```

112         pooled (bool, optional): If True and target_id is None, returns a single
113         concatenated array of all gap times. Defaults to
False.

114
115     Returns:
116         np.ndarray or dict: Array of gap times or a dictionary mapping
117         target index -> numpy array of gap times.
118     """
119     df = self.get_dataframe()
120     if df.empty:
121         return np.array([], dtype=float) if target_id is not None else {}
122
123     # Ensure sorted chronologically
124     df_sorted = df.sort_values('time')
125
126     if target_id is not None:
127         target_df = df_sorted[df_sorted['target_indices'] == target_id]
128         if len(target_df) < 2:
129             return np.array([], dtype=float)
130         time_diffs = target_df['time'].diff().dropna()
131         return time_diffs.dt.total_seconds().values
132     elif pooled:
133         all_gaps = []
134         for tid, group in df_sorted.groupby('target_indices'):
135             if len(group) >= 2:
136                 time_diffs = group['time'].diff().dropna()
137                 all_gaps.extend(time_diffs.dt.total_seconds().tolist())
138         return np.array(all_gaps, dtype=float)
139     else:
140         gaps_dict = {}
141         for tid, group in df_sorted.groupby('target_indices'):
142             if len(group) < 2:
143                 gaps_dict[int(tid)] = np.array([], dtype=float)
144             else:
145                 time_diffs = group['time'].diff().dropna()
146                 gaps_dict[int(tid)] = time_diffs.dt.total_seconds().values
147         return gaps_dict
148
149     def plot_gap_times_histogram(
150         self,
151         target_id: int = None,
152         bins: int | str | None = "auto",
153         show_plot: bool = True
154     ) -> go.Figure:
155         """
156         Generates and optionally displays a Plotly histogram of the target gap times.
157
158         If target_id is specified, plots the histogram of gap times for that target.
159         If target_id is None, plots the histogram of pooled gap times across all targets
160         .
161
162         Args:
163             target_id (int, optional): The index of a specific target. Defaults to None.
164             bins (int or str, optional): Specification of histogram bins (e.g. 'auto',
165             10).
166             Passed to Plotly's nbinsx. Defaults to 'auto'.
167             show_plot (bool, optional): If True, displays the plot. Defaults to True.
168
169         Returns:
170             plotly.graph_objects.Figure: The generated Plotly figure object.
171         """
172         if target_id is not None:
173             gaps = self.calculate_gap_times(target_id=target_id)
174             title = f"Interobservation Gap Times for Target {target_id}"
175         else:
176             gaps = self.calculate_gap_times(pooled=True)
177             title = "Pooled Interobservation Gap Times (All Targets)"
178
179         if len(gaps) == 0:
180             fig = go.Figure()
181             fig.update_layout(
182                 title_text=f"{title} - No Gap Data Available",
183                 template="plotly_white"

```

```

182         )
183         if show_plot:
184             fig.show()
185         return fig
186
187     fig = go.Figure()
188     fig.add_trace(go.Histogram(
189         x=gaps,
190         nbinsx=bins if isinstance(bins, int) else None,
191         marker=dict(
192             color='#3B82F6',
193             line=dict(color='white', width=1)
194         ),
195         opacity=0.85,
196         name="Gap Times "
197     ))
198
199     fig.update_layout(
200         title_text=title,
201         xaxis_title="Gap Time (seconds)",
202         yaxis_title="Observation Count",
203         template="plotly_white",
204         bargap=0.05,
205         hovermode="x unified"
206     )
207
208     if show_plot:
209         fig.show()
210
211     return fig

```

Listing 15.1: dataHandling.py Source Code

Chapter 16

cadenceController.py: Discrete Event Scan Scheduler

Overview

Groups detectors by integration time and schedules discrete integration events.

Literate Source Code

The source code of `cadenceController.py` is documented below. Section label: 16.

```
1 import numpy as np
2 from datetime import timedelta
3 from propagation import propagate_satellites
4 from scandetectors import scandetectors
5 from celestialbodies import celestial_update
6 from minimalsimulation import CadenceGroup, CadenceState
7 from observatories import propagate_observatories
8
9
10 def initCadence(sim_data) -> None:
11     """
12     Initialises sim_data.cadenceStructure from detector integration times.
13
14     Groups detectors that share the same integration time into CadenceGroup
15     objects and stores them in a CadenceState on sim_data.cadenceStructure.
16     All groups are initialised with scanNext equal to the current simulation
17     time so that every group performs a scan on the first call to
18     nextIntegration.
19
20     Args:
21         sim_data: The main SimulationState object.
22     """
23     integration_times = sim_data.detector.integrationTime
24     unique_intervals = np.unique(integration_times)
25
26     cadence_list = [
27         CadenceGroup(
28             scanInterval=float(interval),
29             scanMask=(integration_times == interval),
30             scanNext=sim_data.time,
31         )
32         for interval in unique_intervals
33     ]
34
35     sim_data.cadenceStructure = CadenceState(cadenceList=cadence_list)
36     _update_next_schedule(sim_data)
37
38
39 def nextIntegration(sim_data, print_output: int = 0) -> dict:
40     """
41     Advances the simulation to the next scheduled scan and executes it.
```

```

42
43 Steps:
44 1. Advances sim_data.time to the earliest scheduled group event.
45 2. Propagates all satellites and updates celestial body positions.
46 3. Runs scandetectors for only the active group's masked detectors.
47 4. Updates that group's scanNext by one scanInterval.
48 5. Recomputes which group fires next.
49
50 Args:
51     sim_data:      The main SimulationState object.
52     print_output:  If > 0, scandetectors prints per-detection details.
53
54 Returns:
55     dict: The detection results returned by scandetectors, containing
56           'time', 'sat_indices', 'target_indices', 'signal', 'noise',
57           and 'snr'.
58
59     """
60     cadence = sim_data.cadenceStructure
61
62     # 1. Advance simulation time to the next scheduled event.
63     sim_data.time = cadence.nextTime
64
65     # 2. Propagate all satellites and celestial bodies to the new time.
66     propagate_satellites(sim_data, sim_data.time)
67     celestial_update(sim_data, sim_data.time)
68     propagate_observatories(sim_data, sim_data.time)
69
70     # 3. Run the scan for the active group's detector subset.
71     group = cadence.cadenceList[cadence.nextGroup]
72     results = scandetectors(sim_data, print_output=print_output,
73                             mask=group.scanMask)
74
75     # 4. Schedule this group's next scan.
76     group.scanNext = sim_data.time + timedelta(seconds=group.scanInterval)
77
78     # 5. Find the next overall event across all groups.
79     _update_next_schedule(sim_data)
80
81     return results
82
83 def _update_next_schedule(sim_data) -> None:
84     """
85     Scans all CadenceGroups and updates sim_data.cadenceStructure with
86     the index and datetime of the earliest upcoming scan.
87
88     Args:
89         sim_data: The main SimulationState object.
90
91     """
92     cadence = sim_data.cadenceStructure
93     next_times = [g.scanNext for g in cadence.cadenceList]
94     min_idx = min(range(len(next_times)), key=lambda i: next_times[i])
95     cadence.nextGroup = min_idx
96     cadence.nextTime = cadence.cadenceList[min_idx].scanNext

```

Listing 16.1: cadenceController.py Source Code

Chapter 17

fibonacciSearch.py: Fibonacci Pointing Vector Resorting

Overview

Generates Fibonacci spherical grids and re-orders vectors to minimize slew distances.

Literate Source Code

The source code of `fibonacciSearch.py` is documented below. Section label: 17.

```
1 # fibonacciSearch
2 # this contains the functions necessary to support a set of detectors
3 # that searches the sphere using a fibonacci grid
4
5 import numpy as np
6 import plotly.graph_objects as go
7
8 # def searchStruct(sim_data,detect):
9 #     '''
10 #         creates the data structure for each of the satellite detectors,
11 #         adds the structure to the detector
12 #     '''
13 #     theta = detect[:,IFOV_IDX]/2
14
15 #     # Calculate solid angle
16 #     theta = fov / 2
17 #     solid_angle = 2 * np.pi * (1 - np.cos(theta))
18
19 #     # Calculate grid_points - blow things up by 0.25 for overlap
20 #     grid_points = int(4 * np.pi / solid_angle * 1.25)
21
22 #     # Generate and store the pointing sphere and place in ['pointing_sphers'][n]
23 #     generate_pointing_sphere(sim_data, grid_points)
24
25 def pointing_vectors(n: int) -> np.ndarray:
26     """
27     Generates n equally spaced points on a unit sphere using the Fibonacci lattice
28     algorithm.
29
30     Args:
31         n: The number of points to generate.
32
33     Returns:
34         A NumPy array of shape (n, 3) for the Cartesian coordinates of the points.
35     """
36     if not isinstance(n, int) or n <= 0:
37         raise ValueError("n must be a positive integer.")
38
39     indices = np.arange(0, n, dtype=float) + 0.5
40     z = 1 - 2 * indices / n
41     radius_xy = np.sqrt(1 - z**2)
```



```

41     golden_angle = np.pi * (3. - np.sqrt(5.))
42     theta = golden_angle * indices
43     x = radius_xy * np.cos(theta)
44     y = radius_xy * np.sin(theta)
45
46     unit_vectors = np.stack([x, y, z], axis=1)
47     return unit_vectors
48
49 def resort_vectors_by_proximity(unit_vectors: np.ndarray) -> np.ndarray:
50     """
51     Resorts a list of vectors by making each subsequent vector the closest one
52     in the remaining set to the previous one.
53
54     Args:
55         unit_vectors: A NumPy array of shape (n, 3) representing the vectors.
56
57     Returns:
58         A new NumPy array with the reordered vectors.
59     """
60     if unit_vectors.ndim != 2 or unit_vectors.shape[1] != 3:
61         raise ValueError("unit_vectors array must have shape (n, 3).")
62
63     n_vectors = unit_vectors.shape[0]
64     if n_vectors == 0:
65         return np.array([])
66
67     remaining_indices = list(range(n_vectors))
68     sorted_vectors = np.zeros_like(unit_vectors)
69
70     # Start with the first vector
71     current_index = remaining_indices.pop(0)
72     sorted_vectors[0] = unit_vectors[current_index]
73
74     for i in range(1, n_vectors):
75         last_vector = sorted_vectors[i-1]
76
77         # Vectorized distance computation to all remaining candidates
78         rem_coords = unit_vectors[remaining_indices]
79         dists = np.linalg.norm(rem_coords - last_vector, axis=1)
80
81         # Find index of minimum distance
82         best_idx_in_rem = np.argmin(dists)
83         best_index = remaining_indices[best_idx_in_rem]
84
85         sorted_vectors[i] = unit_vectors[best_index]
86         remaining_indices.pop(best_idx_in_rem)
87
88     return sorted_vectors
89
90
91 def plot_vectors_on_sphere(vectors: np.ndarray, title: str) -> go.Figure:
92     """
93     Creates a 3D plot of vectors on a sphere.
94
95     Args:
96         vectors: A NumPy array of shape (n, 3) representing the vectors.
97         title: The title of the plot.
98
99     Returns:
100         A Plotly figure object.
101     """
102     if vectors.ndim != 2 or vectors.shape[1] != 3:
103         raise ValueError("vectors array must have shape (n, 3).")
104
105     fig = go.Figure()
106
107     fig.add_trace(go.Scatter3d(
108         x=vectors[:, 0], y=vectors[:, 1], z=vectors[:, 2],
109         mode='markers',
110         marker=dict(
111             size=2,
112             color='red',
113             opacity=0.8

```

```

114         ),
115         name='Vectors'
116     ))
117
118     u_sphere = np.linspace(0, 2 * np.pi, 100)
119     v_sphere = np.linspace(0, np.pi, 100)
120     x_earth = 1.0 * np.outer(np.cos(u_sphere), np.sin(v_sphere))
121     y_earth = 1.0 * np.outer(np.sin(u_sphere), np.sin(v_sphere))
122     z_earth = 1.0 * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
123     fig.add_trace(go.Surface(
124         x=x_earth, y=y_earth, z=z_earth,
125         colorscale='Blues', showscale=False, opacity=0.5, name='Sphere'
126     ))
127
128     fig.update_layout(
129         title=title,
130         scene=dict(
131             xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
132             aspectmode='data'
133         ),
134         margin=dict(r=20, b=10, l=10, t=40),
135         legend_title_text='Objects'
136     )
137
138     return fig
139
140 def test_vector_resorting():
141     """
142     Tests the vector resorting and plots the Euclidean distance between subsequent
143     vectors.
144     """
145     # 1. Generate 100 vectors
146     vectors = pointing_vectors(100)
147
148     # 2. Calculate distances without resorting
149     diffs_unsorted = np.linalg.norm(np.diff(vectors, axis=0), axis=1)
150
151     # 3. Resort the vectors
152     sorted_vectors = resort_vectors_by_proximity(vectors)
153
154     # 4. Calculate distances with resorting
155     diffs_sorted = np.linalg.norm(np.diff(sorted_vectors, axis=0), axis=1)
156
157     # 5. Plot the results
158     fig = go.Figure()
159     fig.add_trace(go.Scatter(y=diffs_unsorted, mode='lines', name='Unsorted'))
160     fig.add_trace(go.Scatter(y=diffs_sorted, mode='lines', name='Resorted by Proximity'))
161
162     fig.update_layout(
163         title="Euclidean Distance Between Subsequent Vectors",
164         xaxis_title="Vector Index",
165         yaxis_title="Euclidean Distance"
166     )
167
168     fig.add_annotation(
169         text="Generated by: test_vector_resorting",
170         xref="paper", yref="paper",
171         x=0.5, y=1.01,
172         showarrow=False,
173         font=dict(size=10, color="gray"),
174         xanchor="center", yanchor="bottom"
175     )
176
177     return fig
178
179 if __name__ == '__main__':
180     fig = test_vector_resorting()
181     fig.show()

```

Listing 17.1: fibonacciSearch.py Source Code

Chapter 18

generate_log_spherical_points.py: Logarithmic Radial Distribution

Overview

Generates 3D points with logarithmic radial and uniform angular distribution.

Literate Source Code

The source code of `generate_log_spherical_points.py` is documented below. Section label: 18.

```
1 import numpy as np
2 from datetime import datetime, timezone
3 from plotting_3d import plot_3d_scatter
4 from constants import GEO_RADIUS
5
6
7 def generate_log_spherical_points(
8     num_points: int,
9     inner_radius: float,
10    outer_radius: float,
11    seed: int = None
12 ) -> np.ndarray:
13     """
14     Generates 3D points with logarithmic radial and uniform angular distribution.
15
16     This function creates a point cloud where point distances from the origin are
17     logarithmically spaced. On any given spherical shell, points are distributed
18     uniformly using the Fibonacci lattice method.
19
20     Args:
21         num_points: The total number of points to generate.
22         inner_radius: The minimum distance from the origin (must be positive).
23         outer_radius: The maximum distance from the origin (must be >= inner_radius).
24         seed: An optional integer to seed the random number generator for
25             reproducible shuffling.
26
27     Returns:
28         A NumPy array of shape (num_points, 3) for the Cartesian
29         coordinates.
30     """
31     # Input validation
32     if not isinstance(num_points, int) or num_points <= 0:
33         raise ValueError("num_points must be a positive integer.")
34     if not (isinstance(inner_radius, (int, float)) and inner_radius > 0):
35         raise ValueError("inner_radius must be a positive number.")
36     if not (isinstance(outer_radius, (int, float)) and outer_radius >= inner_radius):
37         raise ValueError("outer_radius must be a positive number and >= inner_radius.")
38
39     # --- 1. Generate uniformly distributed unit vectors (Fibonacci Lattice) ---
40     from fibonacciSearch import pointing_vectors
```

```

42     unit_vectors = pointing_vectors(num_points)
43
44     # --- 2. Generate logarithmically spaced radii ---
45     # The start and stop parameters for logspace are exponents
46     start_exp = np.log10(inner_radius)
47     stop_exp = np.log10(outer_radius)
48     radii = np.logspace(start_exp, stop_exp, num_points)
49
50     # --- 3. Shuffle radii to decouple radius from spherical position ---
51     # This prevents the smallest radii from always mapping to points near the
52     # north pole, creating a more uniform visual distribution of densities.
53     rng = np.random.default_rng(seed)
54     rng.shuffle(radii)
55
56     # --- 4. Scale unit vectors by radii using broadcasting ---
57     # Reshape radii to (N, 1) to broadcast with (N, 3) unit_vectors
58     points = unit_vectors * radii[:, np.newaxis]
59
60     return points
61
62 if __name__ == '__main__':
63     # --- Demo of Point Generation and Visualization ---
64     # This provides a consistent demonstration entry point, matching vibevolts_demo.py
65     NUM_POINTS = 100
66     # Using the same radii as the main simulation for consistency
67     INNER_RADIUS = 2000000
68     OUTER_RADIUS = 2 * GEO_RADIUS
69     OBJECT_SIZE = 1.0
70
71     print(f"Generating {NUM_POINTS} points from radius {INNER_RADIUS} to {OUTER_RADIUS}...")
72     generated_points = generate_log_spherical_points(
73         num_points=NUM_POINTS,
74         inner_radius=INNER_RADIUS,
75         outer_radius=OUTER_RADIUS,
76         seed=42
77     )
78     print("Point generation complete.\n")
79
80     # Use the unified visualization function
81     plot_3d_scatter(
82         positions=generated_points,
83         title="Fixed Points Distribution (from generate_log_spherical_points.py)",
84         plot_time=datetime.now(timezone.utc),
85         labels=[f"Point {i}" for i in range(len(generated_points))],
86         marker_size=1,
87         trace_name='Fixed Points'
88     )

```

Listing 18.1: generate_log_spherical_points.py Source Code

Chapter 19

plotting_3d.py: Plotly 3D Position Renderer

Overview

Renders 3D interactive Plotly figures for satellite, observatory, and target positions.

Literate Source Code

The source code of plotting_3d.py is documented below. Section label: 19.

```
1 import numpy as np
2 from datetime import datetime
3 from typing import List, Optional
4
5 import plotly.graph_objects as go
6 from astropy.time import Time
7 from astropy.coordinates import GCRS, EarthLocation
8 import astropy.units as u
9
10 from constants import EARTH_RADIUS
11
12 def plot_3d_scatter(
13     positions: np.ndarray,
14     title: str,
15     plot_time: datetime,
16     labels: Optional[List[str]] = None,
17     marker_size: int = 1,
18     trace_name: str = 'Points',
19     marker_color: Optional[str] = None
20 ) -> go.Figure:
21     """
22     Creates a 3D plot of object positions.
23     """
24     if positions.ndim != 2 or positions.shape[1] != 3:
25         raise ValueError("positions array must have shape (n, 3).")
26
27     fig = go.Figure()
28
29     marker_dict = dict(
30         size=marker_size,
31         opacity=0.8
32     )
33     if marker_color:
34         marker_dict['color'] = marker_color
35     else:
36         marker_dict['color'] = np.arange(len(positions))
37         marker_dict['colorscale'] = 'Viridis'
38
39     fig.add_trace(go.Scatter3d(
40         x=positions[:, 0], y=positions[:, 1], z=positions[:, 2],
41         mode='markers',
```

```

42     marker=marker_dict,
43     text=labels,
44     hoverinfo='text' if labels else 'none',
45     name=trace_name
46 ))
47
48 u_sphere = np.linspace(0, 2 * np.pi, 100)
49 v_sphere = np.linspace(0, np.pi, 100)
50 x_earth = EARTH_RADIUS * np.outer(np.cos(u_sphere), np.sin(v_sphere))
51 y_earth = EARTH_RADIUS * np.outer(np.sin(u_sphere), np.sin(v_sphere))
52 z_earth = EARTH_RADIUS * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
53 fig.add_trace(go.Surface(
54     x=x_earth, y=y_earth, z=z_earth,
55     colorscale='Blues', showscale=False, opacity=0.5, name='Earth'
56 ))
57
58 theta = np.linspace(0, 2 * np.pi, 100)
59 x_eq = EARTH_RADIUS * np.cos(theta)
60 y_eq = EARTH_RADIUS * np.sin(theta)
61 z_eq = np.zeros_like(theta)
62 fig.add_trace(go.Scatter3d(
63     x=x_eq, y=y_eq, z=z_eq, mode='lines',
64     line=dict(color='green', width=3), name='Equator'
65 ))
66
67 fig.add_trace(go.Scatter3d(
68     x=[0], y=[0], z=[EARTH_RADIUS * 1.1], mode='text',
69     text=['N'], textfont=dict(size=15, color='red'), name='North Pole'
70 ))
71
72 lat_es, lon_es = 33.92 * u.deg, -118.42 * u.deg
73 el_segundo_loc = EarthLocation.from_geodetic(lon=lon_es, lat=lat_es)
74 itrans_coords = el_segundo_loc.get_itrs(obstime=Time(plot_time))
75 gcrs_coords = itrans_coords.transform_to(GCRS(obstime=Time(plot_time)))
76 es_pos = gcrs_coords.cartesian.xyz.to(u.m).value * 1.05
77 fig.add_trace(go.Scatter3d(
78     x=[es_pos[0]], y=[es_pos[1]], z=[es_pos[2]], mode='text',
79     text=['ES'], textfont=dict(size=15, color='yellow'), name='El Segundo'
80 ))
81
82 fig.update_layout(
83     title=title,
84     scene=dict(
85         xaxis_title='X (m)', yaxis_title='Y (m)', zaxis_title='Z (m)',
86         aspectmode='data'
87     ),
88     margin=dict(r=20, b=10, l=10, t=40),
89     legend_title_text='Objects'
90 )
91
92 return fig

```

Listing 19.1: plotting_3d.py Source Code

Chapter 20

plotting_vectors.py: Plotly Pointing Vector Renderer

Overview

Renders 3D interactive Plotly figures for sensor pointing vectors.

Literate Source Code

The source code of `plotting_vectors.py` is documented below. Section label: [20](#).

```
1 import numpy as np
2 from datetime import datetime
3 from typing import Dict, Any
4
5 import plotly.graph_objects as go
6
7 from constants import EARTH_RADIUS
8
9 def plot_pointing_vectors(
10     data_struct: Dict[str, Any],
11     title: str,
12     plot_time: datetime
13 ) -> go.Figure:
14     """
15     Creates a 3D plot of satellites with pointing vectors.
16     """
17     sat_positions = data_struct.satellites.position.copy()
18     sat_pointing = data_struct.detector.pointing.copy()
19     num_sats = data_struct.counts.satellites
20
21     if num_sats == 0:
22         print("No satellites to plot.")
23         return go.Figure()
24
25     fig = go.Figure()
26
27     fig.add_trace(go.Scatter3d(
28         x=sat_positions[:, 0], y=sat_positions[:, 1], z=sat_positions[:, 2],
29         mode='markers',
30         marker=dict(size=5, color='blue', opacity=0.8),
31         name='Satellites'
32     ))
33
34     vector_scale = 0.5 * EARTH_RADIUS
35     for i in range(num_sats):
36         start_point = sat_positions[i]
37         pointing_vec = sat_pointing[i]
38         norm = np.linalg.norm(pointing_vec)
39         unit_vec = pointing_vec / norm if norm > 0 else np.array([0, 0, 0])
40         end_point = start_point + unit_vec * vector_scale
41         fig.add_trace(go.Scatter3d(
```

```

42         x=[start_point[0], end_point[0]],
43         y=[start_point[1], end_point[1]],
44         z=[start_point[2], end_point[2]],
45         mode='lines',
46         line=dict(color='red', width=3),
47         showlegend=False
48     ))
49
50     u_sphere = np.linspace(0, 2 * np.pi, 100)
51     v_sphere = np.linspace(0, np.pi, 100)
52     x_earth = EARTH_RADIUS * np.outer(np.cos(u_sphere), np.sin(v_sphere))
53     y_earth = EARTH_RADIUS * np.outer(np.sin(u_sphere), np.sin(v_sphere))
54     z_earth = EARTH_RADIUS * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
55     fig.add_trace(go.Surface(
56         x=x_earth, y=y_earth, z=z_earth,
57         colorscale='Blues', showscale=False, opacity=0.5, name='Earth'
58     ))
59
60     fig.update_layout(
61         title=title,
62         scene=dict(
63             xaxis_title='X (m)', yaxis_title='Y (m)', zaxis_title='Z (m)',
64             aspectmode='data'
65         ),
66         margin=dict(r=20, b=10, l=10, t=40)
67     )
68
69     return fig

```

Listing 20.1: plotting_vectors.py Source Code

Chapter 21

sim_report.py: Simulation State Reporting

Overview

Prints formatted state reports for satellites, observatories, and exclusion parameters.

Literate Source Code

The source code of `sim_report.py` is documented below. Section label: 21.

```
1 import numpy as np
2 from astropy.coordinates import EarthLocation, GCRS, AltAz, SkyCoord
3 from astropy.time import Time
4 import astropy.units as u
5
6 from constants import (
7     ORBITAL_A_IDX,
8     ORBITAL_E_IDX,
9     ORBITAL_I_IDX,
10    ORBITAL_RAAN_IDX,
11    ORBITAL_ARGP_IDX,
12    ORBITAL_M_IDX
13 )
14
15 def print_simulation_state(sim_data):
16     """
17     Prints the current state of the simulation in tabulated format.
18     Includes satellites (orbital elements, pointing RA/Dec),
19     observatories (lat/lon/alt, pointing Az/El), and exclusion parameters.
20     """
21     print("\n" + "="*80)
22     print("SIMULATION STATE REPORT")
23     print(f"Time: {sim_data.time}")
24     print("="*80)
25
26     # 1. Satellites
27     print("\n--- SATELLITES ---")
28     if 'satellites' in sim_data.counts and sim_data.counts.satellites > 0:
29         num_sats = sim_data.counts.satellites
30         print(f"{'Idx':<4} | {'Semi-Major (m)':<15} | {'Eccentricity':<12} | {'Inc (deg)':<9} | {'RAAN (deg)':<10} | {'Sensor Band':<11} | {'Pointing RA (deg)':<17} | {'Pointing Dec (deg)':<18}")
31         print("-" * 115)
32
33     # Build lookup array for detectors safely if detector is present
34     has_detectors = (sim_data.detector is not None and len(sim_data.detector.filt) > 0)
35     if has_detectors:
36         det_cat = np.array(sim_data.detector.category)
37         det_idx = sim_data.detector.asset_index
38     else:
```

```

39     det_cat, det_idx = np.array([]), np.array([])
40
41     for i in range(num_sats):
42         oe = sim_data.satellites.orbital_elements[i]
43         a = oe[ORBITAL_A_IDX]
44         e = oe[ORBITAL_E_IDX]
45         inc = np.rad2deg(oe[ORBITAL_I_IDX])
46         raan = np.rad2deg(oe[ORBITAL_RAAN_IDX])
47
48         match = np.where((det_cat == 'satellites') & (det_idx == i))[0]
49         if len(match) > 0:
50             d_i = match[0]
51             band = sim_data.detector.filt[d_i]
52             pt = sim_data.detector.pointing[d_i]
53             # Pointing vector is in GCRS
54             ra = np.rad2deg(np.arctan2(pt[1], pt[0])) % 360
55             dec = np.rad2deg(np.arcsin(pt[2]))
56             band_str = band
57             ra_str = f"{ra:.2f}"
58             dec_str = f"{dec:.2f}"
59         else:
60             band_str = "None"
61             ra_str = "N/A"
62             dec_str = "N/A"
63
64         print(f"{i:<4} | {a:<15.2f} | {e:<12.6f} | {inc:<9.2f} | {raan:<10.2f} | {
band_str:<11} | {ra_str:<17} | {dec_str:<18}")
65     else:
66         print("No satellites in simulation.")
67
68     # 2. Observatories
69     print("\n--- OBSERVATORIES ---")
70     if 'observatories' in sim_data.counts and sim_data.counts.observatories > 0:
71         num_obs = sim_data.counts.observatories
72         print(f"{Idx':<4} | {Lat (deg)':<9} | {Lon (deg)':<10} | {Alt (m)':<9} | {'
Sensor Band':<11} | {'Pointing Az (deg)':<17} | {'Pointing El (deg)':<18}")
73         print("-" * 95)
74
75         if has_detectors:
76             det_cat = np.array(sim_data.detector.category)
77             det_idx = sim_data.detector.asset_index
78         else:
79             det_cat, det_idx = np.array([]), np.array([])
80
81         obs_time = Time(sim_data.time)
82
83         for i in range(num_obs):
84             lat = sim_data.observatories.latitude[i]
85             lon = sim_data.observatories.longitude[i]
86             alt = sim_data.observatories.altitude[i]
87
88             match = np.where((det_cat == 'observatories') & (det_idx == i))[0]
89             if len(match) > 0:
90                 d_i = match[0]
91                 band = sim_data.detector.filt[d_i]
92                 pt = sim_data.detector.pointing[d_i]
93
94                 # Pointing vector is in GCRS. Convert to AltAz frame.
95                 loc = EarthLocation(lat=lat*u.deg, lon=lon*u.deg, height=alt*u.m)
96                 coord = SkyCoord(x=pt[0], y=pt[1], z=pt[2], representation_type='
cartesian', frame=GCRS(obstime=obs_time))
97                 altaz = coord.transform_to(AltAz(obstime=obs_time, location=loc))
98
99                 az = altaz.az.deg
100                el = altaz.alt.deg
101
102                band_str = band
103                az_str = f"{az:.2f}"
104                el_str = f"{el:.2f}"
105            else:
106                band_str = "None"
107                az_str = "N/A"
108                el_str = "N/A"

```

```

109         print(f"{i:<4} | {lat:<9.4f} | {lon:<10.4f} | {alt:<9.1f} | {band_str:<11} |
110               {az_str:<17} | {el_str:<18}")
111     else:
112         print("No observatories in simulation.")
113
114     # 3. Exclusion Parameters
115     print("\n--- EXCLUSION PARAMETERS ---")
116     if has_detectors:
117         print(f"{'Category':<15} | {'Asset Idx':<9} | {'Solar Excl (deg)':<16} | {'Lunar
118               Excl (deg)':<16} | {'Earth Excl (deg)':<16}")
119         print("-" * 80)
120
121         for d_i in range(len(sim_data.detector.filt)):
122             cat = sim_data.detector.category[d_i]
123             a_idx = sim_data.detector.asset_index[d_i]
124             solar_ex = np.rad2deg(sim_data.detector.solarEx[d_i])
125             lunar_ex = np.rad2deg(sim_data.detector.lunarEx[d_i])
126             earth_ex = np.rad2deg(sim_data.detector.earthEx[d_i])
127
128             print(f"{cat:<15} | {a_idx:<9} | {solar_ex:<16.2f} | {lunar_ex:<16.2f} | {
129                   earth_ex:<16.2f}")
130         else:
131             print("No detectors configured.")
132
133     print("="*80 + "\n")

```

Listing 21.1: sim.report.py Source Code

Chapter 22

sim_check.py: Simulation Data Verification

Overview

Utility for checking keys, counts, and non-zero initialization of sim_data.

Literate Source Code

The source code of `sim_check.py` is documented below. Section label: [22](#).

```
1 import numpy as np
2 from datetime import datetime, timezone
3 from typing import Any
4
5 def sim_check(sim_data: Any) -> None:
6     """
7     Validates and prints a diagnostic summary of the simulation data structure.
8
9     Checks key properties of the simulation state (such as start time, step time,
10    pointing spheres, celestial bodies, satellites, and detector configurations)
11    and reports warnings if crucial simulation elements are missing or uninitialized.
12
13    Args:
14        sim_data: The main simulation data structure (SimulationState).
15    """
16    print("=====")
17    print("--- Simulation Data Check ---")
18
19    # Check for start_time
20    if hasattr(sim_data, 'start_time') and sim_data.start_time is not None:
21        print(f"        Start Time: {sim_data.start_time}")
22    else:
23        print("*** Start Time: Not found")
24
25    # Check for delta_time
26    if hasattr(sim_data, 'delta_time') and sim_data.delta_time is not None:
27        print(f"        Delta Time: {sim_data.delta_time}")
28    else:
29        print("*** Delta Time: Not found")
30
31    # Check for pointing_spheres
32    if hasattr(sim_data, 'pointing_spheres') and sim_data.pointing_spheres:
33        print(f"        Pointing Sphere Keys: {list(sim_data.pointing_spheres.keys())}")
34    else:
35        print("*** Pointing Spheres: Not found or empty")
36
37    # Check for celestial
38    if hasattr(sim_data, 'celestial') and sim_data.celestial:
39        print(f"        Celestial Dictionary Keys: {list(sim_data.celestial.keys())}")
40    else:
```

```

41     print("*** Celestial Dictionary: Not found or empty")
42
43     # Check for satellites
44     if (hasattr(sim_data, 'satellites') and sim_data.satellites and
45         hasattr(sim_data, 'counts') and sim_data.counts and
46         hasattr(sim_data.counts, 'satellites') and sim_data.counts.satellites):
47         num_sats = sim_data.counts.satellites
48         print(f"        Number of Satellites: {num_sats}")
49
50     if hasattr(sim_data.satellites, 'position') and sim_data.satellites.position is
not None:
51         if np.all(sim_data.satellites.position == 0):
52             print("*** Satellite positions are all zero.")
53         else:
54             print("        Satellite positions are not all zero.")
55     else:
56         print("*** Satellite positions not found.")
57
58     if hasattr(sim_data, 'detector') and sim_data.detector:
59         detector = sim_data.detector
60         for i in range(num_sats):
61             # Check a few attributes to see if they are non-zero
62             if (hasattr(detector, 'apertureArea') and detector.apertureArea[i] != 0
or
63                 hasattr(detector, 'pixelOmega') and detector.pixelOmega[i] != 0 or
64                 hasattr(detector, 'qe') and detector.qe[i] != 0):
65                 print(f"        - Satellite {i} has a detector.")
66             else:
67                 print(f"***        - Satellite {i} does not have a detector.")
68         else:
69             print("*** Detector information not found for satellites.")
70     else:
71         print("*** Satellites: Not found")
72
73     print("--- End of Check ---")
74
75 if __name__ == '__main__':
76     # Create a dummy sim_data for demonstration
77     from detector import makeBlankDetector
78     from minimalsimulation import (
79         create_empty_simulation,
80         SatellitesState,
81         CelestialState,
82     )
83     dummy_detector = makeBlankDetector(2)
84     dummy_detector.apertureArea = np.array([0.785, 0.])
85     dummy_detector.pixelOmega = np.array([1e-10, 0.])
86     dummy_detector.qe = np.array([0.5, 0.5])
87
88     dummy_sim_data = create_empty_simulation(
89         start_time=datetime(2025, 1, 1, 0, 0, 0, tzinfo=timezone.utc),
90         delta_time=60.0
91     )
92     dummy_sim_data.counts.satellites = 2
93     dummy_sim_data.counts.celestial = 2
94     dummy_sim_data.pointing_spheres = {
95         'sphere1': np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]]),
96     }
97     dummy_sim_data.celestial = CelestialState(
98         position=np.zeros((2, 3))
99     )
100     dummy_sim_data.satellites = SatellitesState(
101         position=np.zeros((2, 3))
102     )
103     dummy_sim_data.detector = dummy_detector
104     sim_check(dummy_sim_data)
105
106     print("\n--- Testing with a minimal structure ---")
107     minimal_sim_data = create_empty_simulation(
108         start_time=datetime(2025, 1, 1, 0, 0, 0, tzinfo=timezone.utc)
109     )

```

```
110 sim_check(minimal_sim_data)
```

Listing 22.1: sim.check.py Source Code

Chapter 23

all_demos.py: Demonstration Suite Runner

Overview

Main entry point to execute themed demo suites ('orbits', 'pointing', 'radiometry').

Literate Source Code

The source code of `all_demos.py` is documented below. Section label: [23](#).

```
1 import plotly.graph_objects as go
2 from typing import Union, List, Callable, Optional
3
4 # Import all demo functions
5 from demo1 import demo1
6 from demo2 import demo2
7 from demo3 import demo3
8 from demogeo import demogeo
9 from demo_fixedpoints import demo_fixedpoints
10 from demo_exclusion_table import demo_exclusion_table
11 from demo_pointing_plot import demo_pointing_plot
12 from radiometry_test import demoFixed
13
14 from demo_lambertian import demo_lambertian
15 from demo_sky_scan import demo_sky_scan
16 from demo_pointing_vectors import demo_pointing_vectors
17 from demo_pointing_sequence import demo_pointing_sequence
18 from demo_constellation import demo_constellation
19 from demo_show_geo_search import demo_show_geo_search
20 from demo_requiredIntegrationTime import demo_requiredIntegrationTime
21 from fibonacciSearch import test_vector_resorting
22 from pointing import demo_exclusion_pointing
23 from demo_gap_time_histogram import demo_gap_time_histogram
24 from demo_observatories_only import demo_observatories_only
25
26
27 def demo_vector_resorting_plot() -> go.Figure:
28     """
29     Runs the test_vector_resorting function and returns its figure.
30     """
31     print("\n--- Starting Demo: Vector Resorting ---")
32     return test_vector_resorting()
33
34
35 DEMO_GROUPS = {
36     'orbits': [
37         ('demo1', demo1),
38         ('demo2', demo2),
39         ('demo3', demo3),
40         ('demogeo', demogeo),
41         ('demo_constellation', demo_constellation),
```

```

42         ('demo_observatories_only', demo_observatories_only),
43     ],
44     'pointing': [
45         ('demo_pointing_plot', demo_pointing_plot),
46         ('demo_sky_scan', demo_sky_scan),
47         ('demo_pointing_vectors', demo_pointing_vectors),
48         ('demo_pointing_sequence', demo_pointing_sequence),
49         ('demo_show_geo_search', demo_show_geo_search),
50         ('demo_exclusion_pointing', demo_exclusion_pointing),
51     ],
52     'radiometry': [
53         ('demo_fixedpoints', demo_fixedpoints),
54         ('demo_exclusion_table', demo_exclusion_table),
55         ('demo_lambertian', demo_lambertian),
56         ('demo_requiredIntegrationTime', demo_requiredIntegrationTime),
57         ('demo_vector_resorting_plot', demo_vector_resorting_plot),
58         ('demoFixed', demoFixed),
59         ('demo_gap_time_histogram', demo_gap_time_histogram),
60     ],
61 }
62
63
64 def list_demo_groups() -> None:
65     """Lists available demo groups and their included demo functions."""
66     print("=== VibeVolts Demo Groups ===")
67     for group_name, demo_list in DEMO_GROUPS.items():
68         print(f"\nGroup: '{group_name}' ({len(demo_list)} demos)")
69         for name, func in demo_list:
70             print(f"    - {name}")
71     print("\nUse run_all_demos(group='<name>') to run a specific group inline.")
72
73
74 def run_all_demos(
75     group: str = 'all',
76     demos: Optional[List[Union[str, Callable]]] = None,
77     save_html: bool = False
78 ) -> None:
79     """
80     Runs demo functions by suite group, specific list, or all demos.
81
82     Args:
83         group (str): Demo group to run: 'orbits', 'pointing', 'radiometry', or 'all'.
84         demos (list): Optional list of demo names or functions to run.
85         save_html (bool): If True, saves plots to HTML file.
86     """
87     funcs_to_run = []
88
89     if demos is not None:
90         all_name_map = {}
91         for g_list in DEMO_GROUPS.values():
92             for name, func in g_list:
93                 all_name_map[name] = func
94         for item in demos:
95             if callable(item):
96                 funcs_to_run.append((item.__name__, item))
97             elif isinstance(item, str) and item in all_name_map:
98                 funcs_to_run.append((item, all_name_map[item]))
99             else:
100                 print(f"Warning: Demo '{item}' not recognized. Skipping.")
101     elif group in DEMO_GROUPS:
102         funcs_to_run = DEMO_GROUPS[group]
103     elif group == 'all':
104         for g_list in DEMO_GROUPS.values():
105             funcs_to_run.extend(g_list)
106     if not save_html:
107         print("Notice: Displaying all demos inline can exceed browser WebGL limits (16 max).")
108         print("Defaulting to save_html=True. Set group='orbits', 'pointing', or 'radiometry' for inline notebook display.")
109         save_html = True
110     else:
111         print(f"Unknown group '{group}'. Available groups: {list(DEMO_GROUPS.keys())} or 'all'.")

```



```

112         return
113
114     figs = []
115     print(f"--- Running Demos (Group: {group}, Count: {len(funcs_to_run)}) ---")
116     for name, func in funcs_to_run:
117         print(f"\n\033[91m--- Executing {name} ---\033[0m")
118         res = func()
119         if isinstance(res, go.Figure):
120             figs.append(res)
121         elif isinstance(res, tuple):
122             for item in res:
123                 if isinstance(item, go.Figure):
124                     figs.append(item)
125
126     if save_html:
127         filename = "all_demo_plots.html" if group == 'all' else f"demo_plots_{group}.html"
128         with open(filename, "w") as f:
129             f.write("<html><head><title>VibeVolts Demos</title></head><body>")
130             f.write(f"<h1>VibeVolts Demo Plots ({group.capitalize()})</h1>")
131             for i, fig in enumerate(figs):
132                 title = fig.layout.title.text if fig.layout.title else f"Plot {i+1}"
133                 f.write(f"<h2>{title}</h2>")
134                 f.write(fig.to_html(full_html=False, include_plotlyjs='cdn'))
135             f.write("</body></html>")
136         print(f"\n--- Demos complete. {len(figs)} plots saved to {filename} ---")
137     else:
138         print(f"\n--- Displaying {len(figs)} demo plots inline ---")
139         for fig in figs:
140             fig.show()
141
142
143 if __name__ == '__main__':
144     run_all_demos()

```

Listing 23.1: all_demos.py Source Code

Index

`all_demos.py`, [69](#)

`cadenceController.py`, [52](#)
`celestialbodies.py`, [21](#)
`constants.py`, [9](#)

`dataHandling.py`, [48](#)
`detector.py`, [10](#)

`exclusion.py`, [24](#)

`fibonacciSearch.py`, [54](#)

`generate_log_spherical_points.py`, [57](#)

`lambertian.py`, [44](#)

`minimalsimulation.py`, [3](#)

`observatories.py`, [18](#)

`plotting_3d.py`, [59](#)
`plotting_vectors.py`, [61](#)
`pointing.py`, [28](#)
`propagation.py`, [15](#)

`radiometry_calcs.py`, [41](#)
`radiometry_data.py`, [38](#)

`scandetectors.py`, [33](#)
`sim_check.py`, [66](#)
`sim_report.py`, [63](#)

`targets.py`, [23](#)